

Otonom Sürüş Teknolojileri Uzmanlık Programı

Sensör Füzyonu ve Durum Kestirimi

Bitirme Projesi

**HAZIRLAYAN
MERT NACİ AKALIN**

İÇİNDEKİLER

1. VERİLEN ROTAYA GÖRE ARACIN HAREKETİNİ ÇIKARTILMASI
2. SENSÖR MODELİ OLUŞTURULMASI
 - A. KALİBRASYON GEREKECEK ŞEKİLDE SENSÖR MODELİNİ BOZULMASI
 - B. SENSOR MODELİNE GÜRÜLTÜ, BİAS VE SCALE FACTOR GİBİ BOZUCU ETKİLER EKLENMESİ
3. GNSS SENSÖRÜNE VERİLEN ZAMANLARDAKİ BOZUCULARI EKLENMESİ
4. SENSÖR KALİBRASYONU YAPILMASI
5. FİLTRELEME TEKNİKLERİ İLE SENSÖR FÜZYONU YAPILMASI
6. ÇIKTILARI GRAFİKLER ÜZERİNDEN ÇİZİLMESİ

ADIM 1

VERİLEN ROTAYA GÖRE ARACIN HAREKETİNİ ÇIKARTILMASI

```
rota.py x
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # - PARAMETRELERİN TANIMLANMASI -
5 # Sistemin çalışma frekansı
6 DT = 1 / 50 #50 Hz çalışacak şekilde örnekleme zamanı
7 TOTAL_TIME = 1000
8 NUM_STEPS = int(TOTAL_TIME / DT) # Toplam adım sayısı 1000 x 0.2 = 50000 adım
9
10 # Başlangıç Koşulları
11 x = 0.0 # metre
12 y = 0.0 # metre
13 vx = 0.0 # m/s
14 vy = 0.0 # m/s
15 yaw = np.radians(90.0) # Radyan (90 derece Kuzey'e bakıyor)
16
17 # Verileri depolamak için listeler
18 time_data = []
19 x_data = []
20 y_data = []
21 vx_data = []
22 vy_data = []
23 yaw_data = []
24 speed_data = [] # Toplam hız (büyüklüğü)
25
26 # - YARDIMCI FONKSİYONLAR -
27 def kmh_to_ms(kmh): 10 usages
28     return kmh / 3.6
29
30 def ms_to_kmh(ms): 4 usages
31     return ms * 3.6
32
33 # - ROTA SİMÜLASYONU DÖNGÜSÜ -
34 current_time = 0.0 # Zamanı takip etmek için bir değişken
35
36 for i in range(NUM_STEPS):
37     # Verileri kaydet
38     time_data.append(current_time)
39     x_data.append(x)
40     y_data.append(y)
41     vx_data.append(vx)
42     vy_data.append(vy)
43     yaw_data.append(np.degrees(yaw)) # Derece olarak kaydet
44     speed_data.append(np.sqrt(vx ** 2 + vy ** 2)) # Hızın büyüklüğü
45
46     # Mevcut hızın büyüklüğü (km/saat)
47     current_speed_kmh = ms_to_kmh(np.sqrt(vx ** 2 + vy ** 2))
48
49     # - ROTA AŞAMALARI -
50     # Aşama 1: Başlangıçtan 10 saniyede 100 km/sa hıza çıkış
51     if current_time <= 10.0:
52         target_speed_ms = kmh_to_ms(100)
53         # Hızlanma ivmesi
54         ax = (target_speed_ms / 10.0) * np.cos(yaw) # ivme = hedef hız / süre
55         ay = (target_speed_ms / 10.0) * np.sin(yaw)
56         angular_velocity = 0.0 # Açısal hız
57
58     # Aşama 2: Sonraki 10 saniye boyunca 45 derece sağa dönüş
59     elif 10.0 < current_time <= 20.0: # 10.0 saniyeden sonraki 10 saniye
60         target_speed_ms = kmh_to_ms(100) # Hız sabit kalacak
61         ax = 0.0
62         ay = 0.0
63         # Açısal hız: 45 derece / 10 saniye
64         angular_velocity = np.radians(-45) / 10.0 # Sağa dönüş negatif açısal hız
```

```
rota.py x
66 # Aşama 3: 200. saniyede hızını 30 saniyede 150 km/s hıza çıkaracak
67 elif 200.0 < current_time <= 230.0: # 200.0 ile 230.0 saniye arası
68     start_speed_kmh = ms_to_kmh(np.sqrt(vx ** 2 + vy ** 2)) # Mevcut hız
69     target_speed_kmh = 150
70     # Hızlanma ivmesi
71     # delta_v_kmh = target_speed_kmh - start_speed_kmh
72     avg_accel_magnitude = (kmh_to_ms(target_speed_kmh) - kmh_to_ms(current_speed_kmh)) / 30.0
73     ax = avg_accel_magnitude * np.cos(yaw)
74     ay = avg_accel_magnitude * np.sin(yaw)
75     angular_velocity = 0.0 # Düz gidiyor
76
77 # Aşama 4: Sonraki 30 saniye boyunca 180 derece dönüş yapacak ve hızını 150 km/s'den 100 km/s'ye indirecek
78 elif 230.0 < current_time <= 260.0: # 230.0 ile 260.0 saniye arası
79     start_speed_kmh = 150
80     end_speed_kmh = 100
81     avg_accel_magnitude = (kmh_to_ms(end_speed_kmh) - kmh_to_ms(start_speed_kmh)) / 30.0
82     ax = avg_accel_magnitude * np.cos(yaw)
83     ay = avg_accel_magnitude * np.sin(yaw)
84     # Açısal hız: 180 derece / 30 saniye
85     # (yön belirtilmemiş ben de sağa doğru döndüğünü varsaydım bu yüzden açısal hızı negatif aldım)
86     angular_velocity = np.radians(-180) / 30.0 # Sağa dönüş
87
88 # Aşama 5: 500. saniyede hızını 10 saniyede 50 km/s hıza indirecek
89 elif 500.0 < current_time <= 510.0: # 500.0 ile 510.0 saniye arası
90     start_speed_kmh = ms_to_kmh(np.sqrt(vx ** 2 + vy ** 2))
91     target_speed_kmh = 50
92     avg_accel_magnitude = (kmh_to_ms(target_speed_kmh) - kmh_to_ms(start_speed_kmh)) / 10.0
93     ax = avg_accel_magnitude * np.cos(yaw)
94     ay = avg_accel_magnitude * np.sin(yaw)
95     angular_velocity = 0.0 # Düz gidiyor
96
```

```
rota.py x
97 # Aşama 6: 800. saniyede 30 saniye boyunca 90 derece sağa dönerken hızını 100 km/saate çıkaracak
98 elif 800.0 < current_time <= 830.0: # 800.0 ile 830.0 saniye arası
99     start_speed_kmh = ms_to_kmh(np.sqrt(vx ** 2 + vy ** 2))
100     target_speed_kmh = 100
101     avg_accel_magnitude = (kmh_to_ms(target_speed_kmh) - kmh_to_ms(start_speed_kmh)) / 30.0
102     ax = avg_accel_magnitude * np.cos(yaw)
103     ay = avg_accel_magnitude * np.sin(yaw)
104
105     # Açısal hız: 90 derece / 30 saniye
106     angular_velocity = np.radians(-90) / 30.0 # Sağa dönüş
107
108 # Aşama 7: 1000. saniyeye son 10 saniye kaldığında araç duracak
109 elif TOTAL_TIME - 10.0 <= current_time < TOTAL_TIME: # 990.0 ile 1000.0 saniye arası
110     start_speed_ms = np.sqrt(vx ** 2 + vy ** 2)
111     # Hedef hız 0, süre 10 saniye
112     avg_decel_magnitude = (0 - start_speed_ms) / 10.0
113     ax = avg_decel_magnitude * np.cos(yaw)
114     ay = avg_decel_magnitude * np.sin(yaw)
115     angular_velocity = 0.0
116
117 # Aşama 8: Diğer zamanlarda hız ve yönelim sabit kalacak (yani ivme ve açısal hız 0)
118 else:
119     ax = 0.0
120     ay = 0.0
121     angular_velocity = 0.0
122     # Hızın neredeyse tam 0 olması ve aracın durması için (float sayılarda sıkıntı olabiliyor)
123     if current_time >= TOTAL_TIME - 10.0 and np.sqrt(vx ** 2 + vy ** 2) < 0.1: # Neredeyse durduysa
124         ax = 0.0
125         ay = 0.0
126         vx = 0.0
127         vy = 0.0
128
```

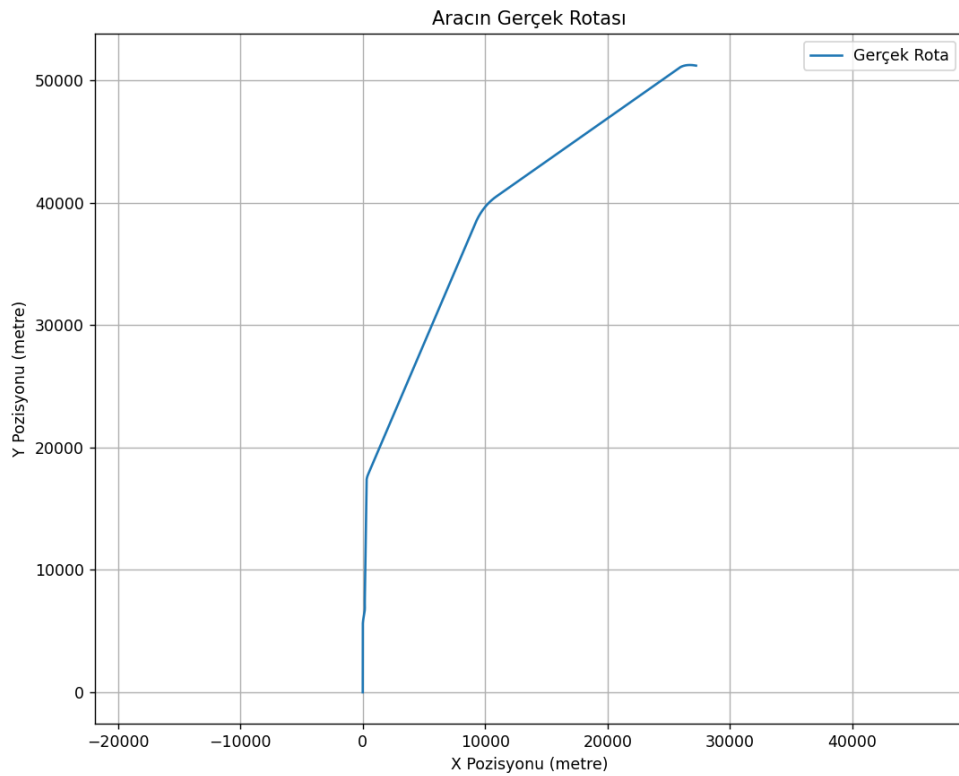
```
rota.py x
129     # Euler İntegrasyonu
130     # Hız güncelleme
131     vx += ax * DT
132     vy += ay * DT
133
134     # Pozisyon güncelleme
135     x += vx * DT
136     y += vy * DT
137
138     # Yönelim (yaw) güncelleme
139     yaw += angular_velocity * DT
140
141     # Zamanı ilerlet
142     current_time += DT
143
144     # Son verileri kaydet (döngü bittikten sonraki son hali)
145     time_data.append(current_time)
146     x_data.append(x)
147     y_data.append(y)
148     vx_data.append(vx)
149     vy_data.append(vy)
150     yaw_data.append(np.degrees(yaw))
151     speed_data.append(np.sqrt(vx ** 2 + vy ** 2))
```

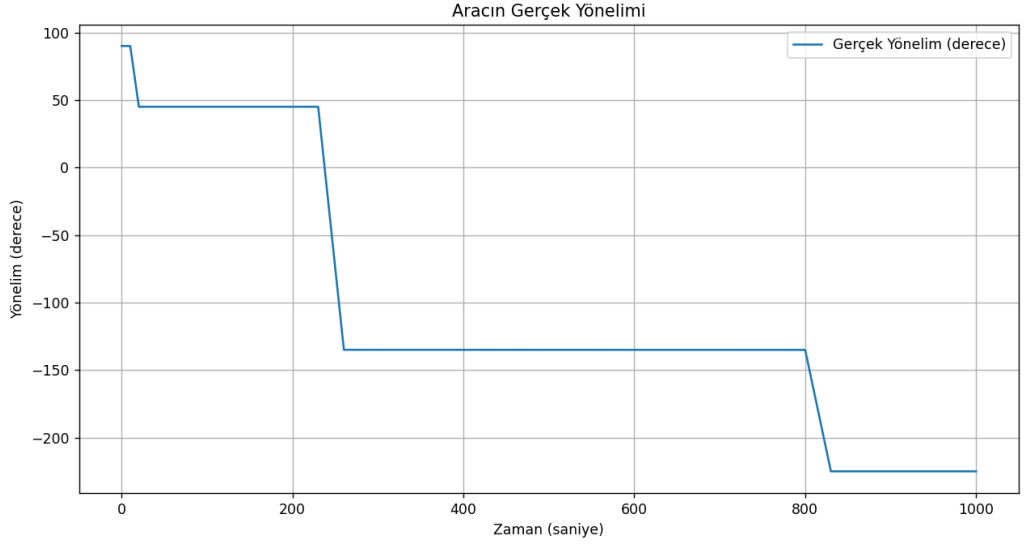
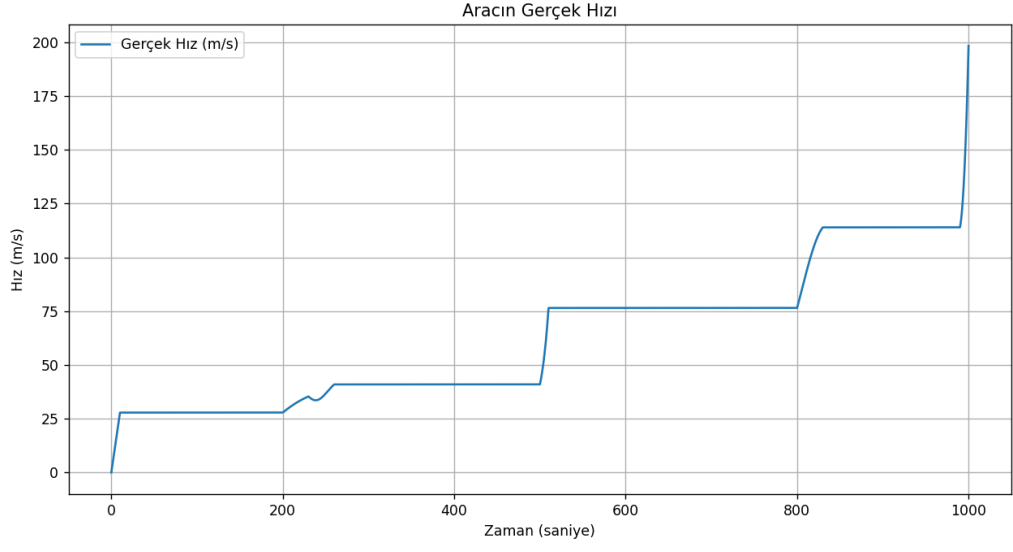
```
rota.py x
153     # - SONUÇLARI GÖRSELLEŞTİRME (ROTA ÇİZİMİ) -
154     plt.figure(figsize=(10, 8))
155     plt.plot(*args: x_data, y_data, label='Gerçek Rota')
156     plt.xlabel('X Pozisyonu (metre)')
157     plt.ylabel('Y Pozisyonu (metre)')
158     plt.title('Aracın Gerçek Rotası')
159     plt.grid(True)
160     plt.axis('equal') # Eksen oranlarını eşit tutar
161     plt.legend()
162     plt.show()
163
164     # Hız grafiği
165     plt.figure(figsize=(12, 6))
166     plt.plot(*args: time_data, speed_data, label='Gerçek Hız (m/s)')
167     plt.xlabel('Zaman (saniye)')
168     plt.ylabel('Hız (m/s)')
169     plt.title('Aracın Gerçek Hızı')
170     plt.grid(True)
171     plt.legend()
172     plt.show()
173
174     # Yönelim grafiği
175     plt.figure(figsize=(12, 6))
176     plt.plot(*args: time_data, yaw_data, label='Gerçek Yönelim (derece)')
177     plt.xlabel('Zaman (saniye)')
178     plt.ylabel('Yönelim (derece)')
179     plt.title('Aracın Gerçek Yönelimi')
180     plt.grid(True)
181     plt.legend()
182     plt.show()
```

```
rota.py x
183
184 # Bu verileri daha sonra kullanmak üzere kaydedelim
185 np.savez(file='true_state_data.npz',
186         time=np.array(time_data),
187         x=np.array(x_data),
188         y=np.array(y_data),
189         vx=np.array(vx_data),
190         vy=np.array(vy_data),
191         yaw=np.array(yaw_data),
192         speed=np.array(speed_data))
193
194 print(f"Rota simülasyonu tamamlandı. Toplam adım sayısı: {NUM_STEPS}")
195 print(f"Son pozisyon: ({x_data[-1]:.2f} m, {y_data[-1]:.2f} m)")
196 print(f"Son hız: {speed_data[-1]:.2f} m/s")
197 print(f"Son yönelim: {yaw_data[-1]:.2f} derece")

Run rota x
C:\Users\ASUS\Desktop\Python\SensorFusion\.venv\Scripts\python.exe C:\Users\ASUS\Desktop\Python\SensorFusion\rota.py
Rota simülasyonu tamamlandı. Toplam adım sayısı: 50000
Son pozisyon: (27220.84 m, 51197.59 m)
Son hız: 198.41 m/s
Son yönelim: -225.00 derece
Process finished with exit code 0
```

Rota simülasyonu tamamlandı. Toplam adım sayısı: 50000 ,
Son pozisyon: (27220.84 m, 51197.59 m),
Son hız: 198.41 m/s, Son yönelim: -225.00 derece





ADIM 2

SENSÖR MODELİ OLUŞTURULMASI

```
rota.py  Sensor.py ×
1  import numpy as np
2  import matplotlib.pyplot as plt
3  # rotadaki verileri yükle
4  loaded_data = np.load('true_state_data.npz')
5  time_true = loaded_data['time']
6  x_true = loaded_data['x']
7  y_true = loaded_data['y']
8  vx_true = loaded_data['vx']
9  vy_true = loaded_data['vy']
10 yaw_true = loaded_data['yaw']
11 speed_true = loaded_data['speed']
12
13 # İvme ve Açılma hızı doğrudan rota simülasyonundan alınmadığı için türevini almamız gerekecek
14 # İvme (a_x_true, a_y_true)
15 DT = time_true[1] - time_true[0] # Zaman adımını tekrar alalım
16 ax_true = np.diff(vx_true) / DT
17 ay_true = np.diff(vy_true) / DT
18 # Diff işlemi dizinin boyutunu 1 kısaltır, son elemanı tekrar ettim
19 ax_true = np.append(ax_true, ax_true[-1]) # Son elemanı tekrarladım
20 ay_true = np.append(ay_true, ay_true[-1])
21
22 # Açılma hızı (yaw rate)
23 # Yaw değeri dereceden radyana çevrilmişti, diff almadan önce radyana çevirelim
24 yaw_rad_true = np.radians(yaw_true)
25 angular_velocity_true = np.diff(yaw_rad_true) / DT
26 angular_velocity_true = np.append(angular_velocity_true, angular_velocity_true[-1])
27 angular_velocity_true_deg = np.degrees(angular_velocity_true) # Derece/saniye olarak kaydetmek için
28
29 # Toplam adım sayısı
30 NUM_STEPS = len(time_true)
31 # DT zaten yukarıda tanımlı
32
33 # - SENSÖR PARAMETRELERİ -
34 # IMU (İvmeölçer ve Jiroskop)
35 # Gürültü (Standart sapma - Örnek değerler, gerçek sensör datasheet'inden alınacak)
36 ACCEL_NOISE_STD = 1 # m/s^2 (İvme ölçer gürültüsü)
37 GYRO_NOISE_STD = np.radians(0.5) # rad/s (jiroskop gürültüsü, 0.5 derece/s)
38
```

```
rota.py  Sensor.py ×
39 # Bias (Sapma - Sensörün sıfır okuması olması gerektiğinde verdiği sabit hata)
40 ACCEL_BIAS_X = 0.2 # m/s^2
41 ACCEL_BIAS_Y = 0.1 # m/s^2
42 GYRO_BIAS = np.radians(0.2) # rad/s (0.2 derece/s)
43
44 # Scale Factor (Ölçek Faktörü - Sensörün gerçek değeri yanlış ölçeklendirmesi)
45 ACCEL_SCALE_X = 1.01
46 ACCEL_SCALE_Y = 0.99
47 GYRO_SCALE = 1.05
48
49 # GNSS (GPS)
50 GNSS_POS_NOISE_STD = 10.0 # metre (Pozisyon gürültüsü)
51 GNSS_VEL_NOISE_STD = 1.5 # m/s (Hız gürültüsü)
52
53 GNSS_UPDATE_RATE = 5 # Hz
54 GNSS_UPDATE_INTERVAL = int(1 / (GNSS_UPDATE_RATE * DT)) # GNSS kaç DT'de bir güncelleme yapacak (50Hz / 5Hz = 10)
55 GNSS_DELAY_SECONDS = 0.2 # saniye (200 milisaniye gecikme)
56 GNSS_DELAY_STEPS = int(GNSS_DELAY_SECONDS / DT) # Kaç adım gecikme olacak
57
58 # GNSS sabit veri üretme (flatline) durumu için global flag ve değerler
59 _gnss_flatline_active = False
60 _flatline_x_val = 0.0
61 _flatline_y_val = 0.0
62 _flatline_vx_val = 0.0
63 _flatline_vy_val = 0.0
64
65 # Sensör verilerini depolamak için listeler
66 imu_accel_x_meas = []
67 imu_accel_y_meas = []
68 imu_gyro_z_meas = [] # Jiroskop Z eksenini (Yaw için)
69
70 gnss_x_meas = []
71 gnss_y_meas = []
72 gnss_vx_meas = []
73 gnss_vy_meas = []
74 gnss_time_meas = [] # GNSS ölçüm zamanları
75
```



```

76 # - SENSÖR SİMÜLASYONU DÖNGÜSÜ -
77 for i in range(NUM_STEPS):
78     # - IMU SİMÜLASYONU -
79     # Hızların türevini alarak ivmeyi ve yaw'ın türevini alarak açısal hızı hesaplıcam
80     # IMU ölçüm modeli: ölçülen = (gerçek + bias) * scale_factor + gürültü
81     # İvmeölçer gerçek ivme değerlerine bias eklencek ve scale factor eklenerek sonra gürültü(0 ile 0.5 arasında) eklenecek
82     imu_x_accel = (ax_true[i] + ACCEL_BIAS_X) * ACCEL_SCALE_X + np.random.normal(loc=0, ACCEL_NOISE_STD)
83     imu_y_accel = (ay_true[i] + ACCEL_BIAS_Y) * ACCEL_SCALE_Y + np.random.normal(loc=0, ACCEL_NOISE_STD)
84
85     # Jiroskop - Z eksenini (Yaw rate)
86     # Jiroskop gerçek açısal hız gürültü, bias ve scale factor eklenerek modelleneyecektir.
87     imu_gyro = (angular_velocity_true[i] + GYRO_BIAS) * GYRO_SCALE + np.random.normal(loc=0, GYRO_NOISE_STD)
88
89     imu_accel_x_meas.append(imu_x_accel)
90     imu_accel_y_meas.append(imu_y_accel)
91     imu_gyro_z_meas.append(imu_gyro)
92
93     # -GNSS SİMÜLASYONU-
94     # GNSS pozisyon ve hız çıktıları üretecektir. Belirlenen frekansta ve gecikmeyle çalışacaktır.
95     if i % GNSS_UPDATE_INTERVAL == 0:
96         # Gecikme uygulanacak olan anın indeksi
97         delay_index = i - GNSS_DELAY_STEPS
98         # Gecikme indeksi 0'dan küçükse (simülasyonun başıysa), gecikme uygulanmaz, mevcut anın verisi alınır
99         if delay_index < 0:
100             delayed_x = x_true[i]
101             delayed_y = y_true[i]
102             delayed_vx = vx_true[i]
103             delayed_vy = vy_true[i]
104         else:
105             delayed_x = x_true[delay_index]
106             delayed_y = y_true[delay_index]
107             delayed_vx = vx_true[delay_index]
108             delayed_vy = vy_true[delay_index]
109

```

```

110 # - GNSS Sensörün Ek Bozuklukları (Adım 3 Gereklilikleri) -
111 current_time = time_true[i]
112
113 # 1. 300. ve 310. saniyeler arasında çıktı üretmeme
114 if 300.0 <= current_time <= 310.0:
115     continue # Bu adımda GNSS ölçümü yapılmaz, sonraki iterasyona geç
116
117 # 2. 400. saniyede 500 metre hatalı gösterme ve 1 saniye sonra düzeltme
118 if 400.0 <= current_time < (400.0 + 1.0):
119     delayed_x += 500.0
120     delayed_y += 500.0

```

```

122 # 3. 500. saniyede 5 saniye boyunca sabit veri üretme
123 if 500.0 <= current_time < 505.0:
124     if not _gnss_flatline_active:
125         # Sabitleme ilk kez başlıyor, o anki gecikmeli gerçek değeri kaydet
126         _flatline_x_val = delayed_x
127         _flatline_y_val = delayed_y
128         _flatline_vx_val = delayed_vx
129         _flatline_vy_val = delayed_vy
130         _gnss_flatline_active = True
131
132     # Sabitlenmiş değerleri kullan
133     delayed_x = _flatline_x_val
134     delayed_y = _flatline_y_val
135     delayed_vx = _flatline_vx_val
136     delayed_vy = _flatline_vy_val
137     else:
138         # Sabitleme süresi dışındaysa flag'i sıfırla
139         _gnss_flatline_active = False
140
141     # GNSS ölçüm modeli: ölçülen = gerçek (gecikmeli) + gürültü (ek bozucular uygulandıktan sonra)
142     gnss_x_m = delayed_x + np.random.normal(loc=0, GNSS_POS_NOISE_STD)
143     gnss_y_m = delayed_y + np.random.normal(loc=0, GNSS_POS_NOISE_STD)
144     gnss_vx_m = delayed_vx + np.random.normal(loc=0, GNSS_VEL_NOISE_STD)
145     gnss_vy_m = delayed_vy + np.random.normal(loc=0, GNSS_VEL_NOISE_STD)
146
147     gnss_x_meas.append(gnss_x_m)
148     gnss_y_meas.append(gnss_y_m)
149     gnss_vx_meas.append(gnss_vx_m)
150     gnss_vy_meas.append(gnss_vy_m)
151     gnss_time_meas.append(time_true[i]) # GNSS ölçümünün yapıldığı zaman

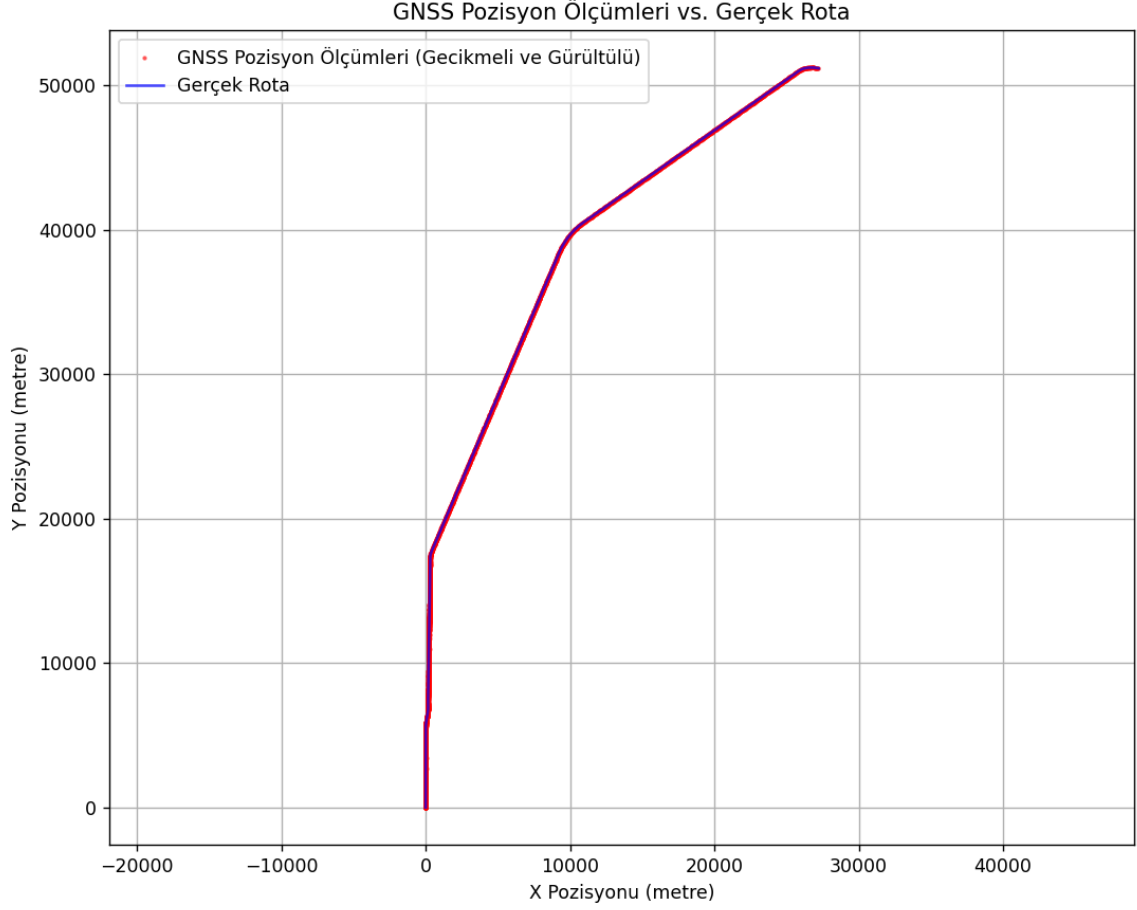
```

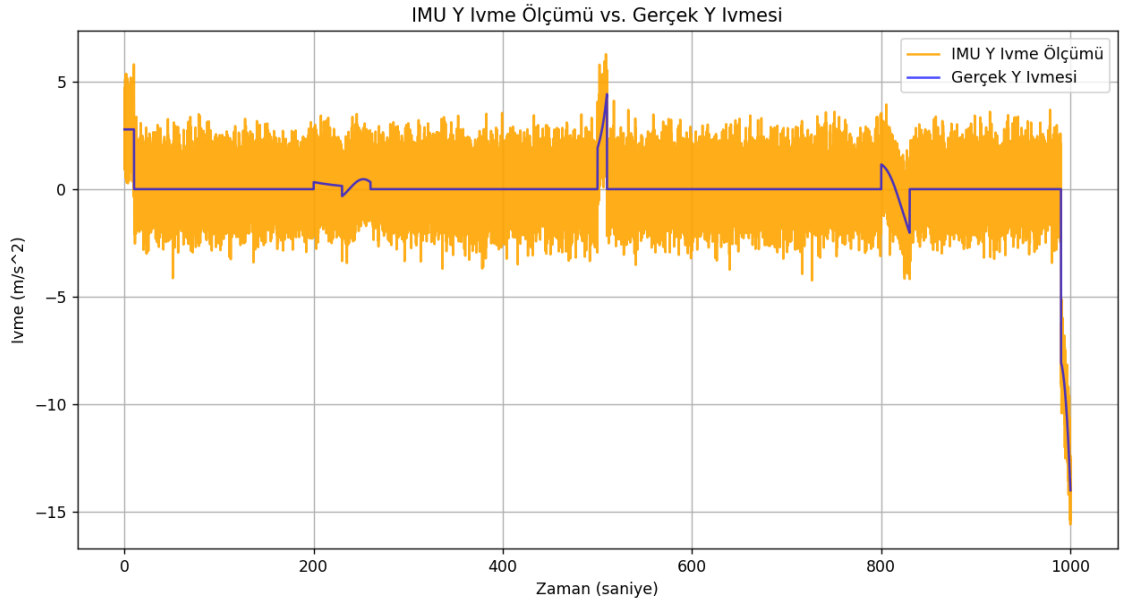
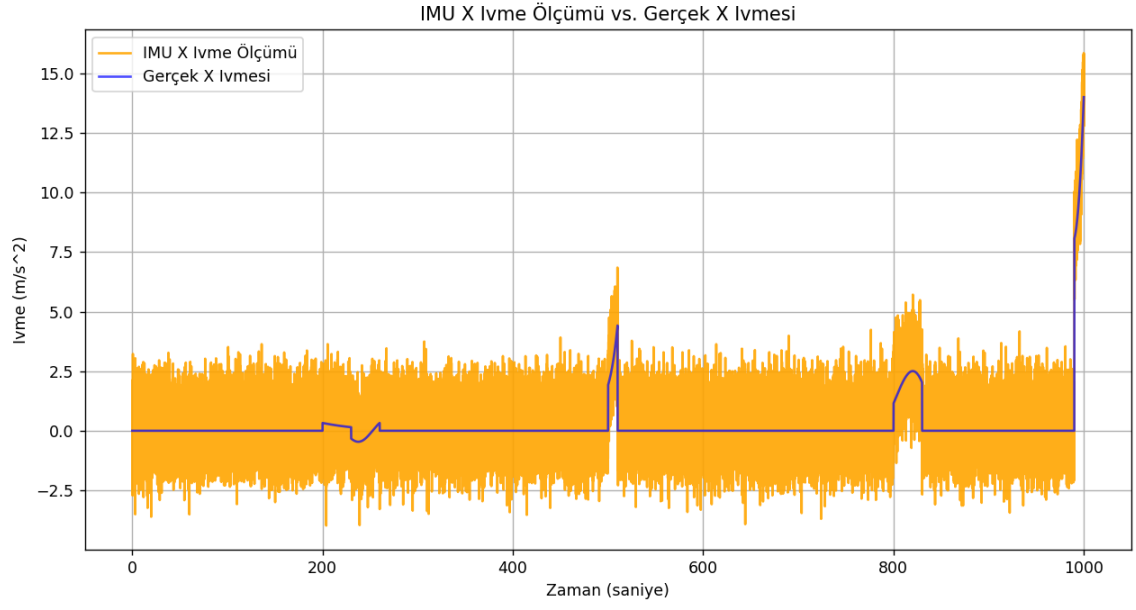
```

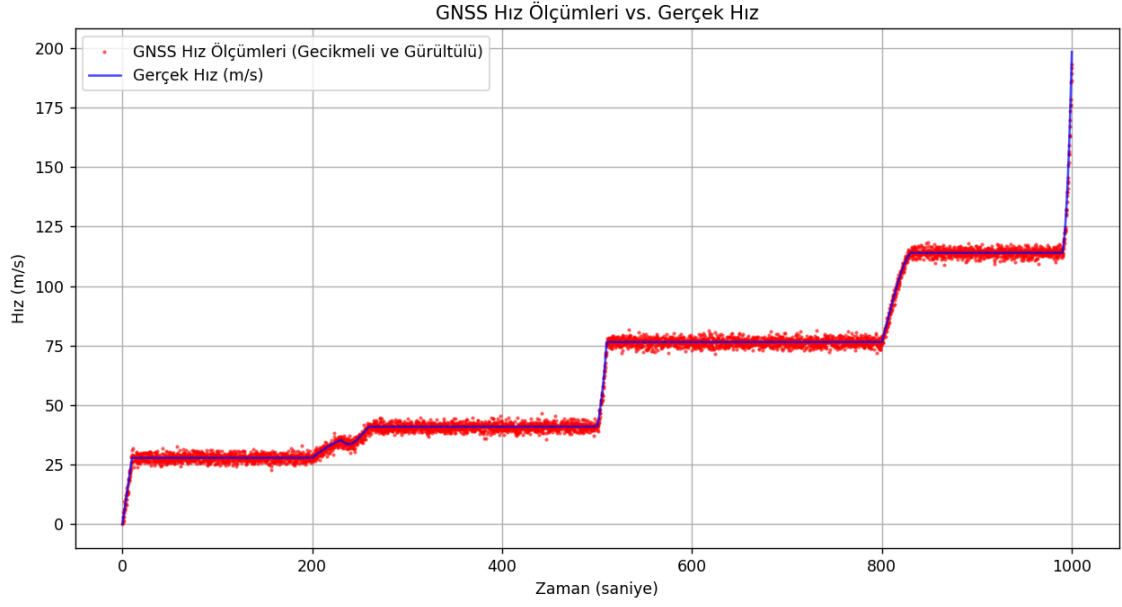
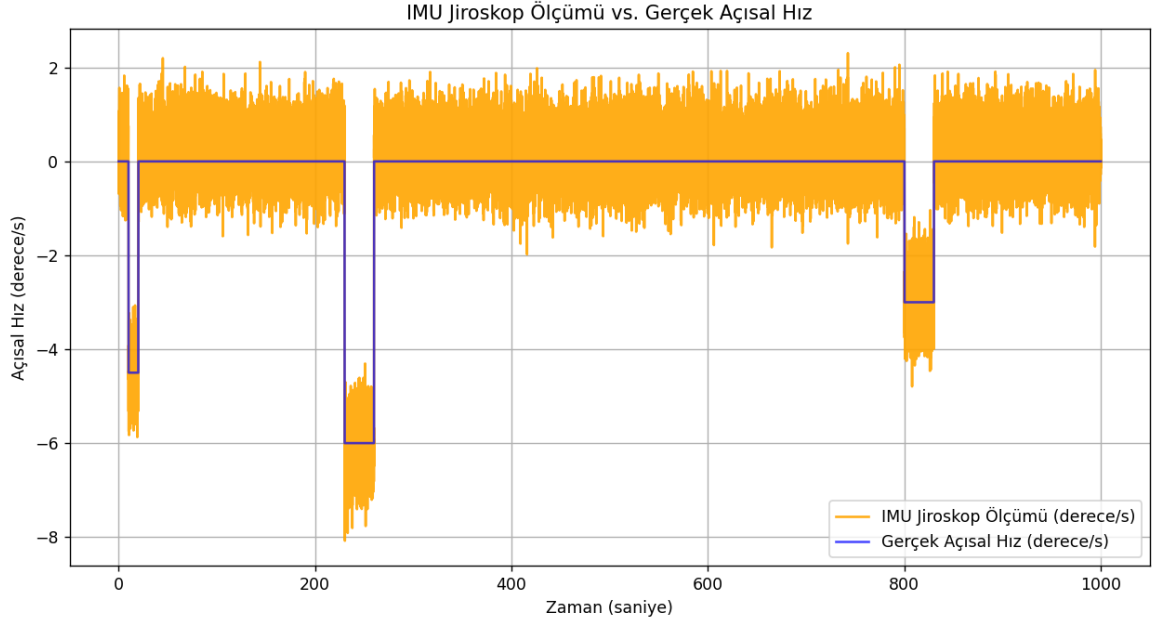
153 # -SENSÖR ÇIKTILARINI GÖRSELLEŞTİRME-
154 # GNSS Pozisyonu (Gerçek Rota ile Karşılaştırma)
155 plt.figure(figsize=(10, 8))
156 plt.plot(*args: gnss_x_meas, gnss_y_meas, 'r.', label='GNSS Pozisyon Ölçümleri (Gecikmeli ve Gürültülü)', alpha=0.5,
157         markersize=3)
158 plt.plot(*args: x_true, y_true, color='blue', label='Gerçek Rota', alpha=0.7)
159 plt.xlabel('X Pozisyonu (metre)')
160 plt.ylabel('Y Pozisyonu (metre)')
161 plt.title('GNSS Pozisyon Ölçümleri vs. Gerçek Rota')
162 plt.grid(True)
163 plt.axis('equal')
164 plt.legend()
165 plt.show()
166 # IMU İvme (Gerçek İvme ile Karşılaştırma)
167 plt.figure(figsize=(12, 6))
168 plt.plot(*args: time_true, imu_accel_x_meas, color='orange', label='IMU X İvme Ölçümü', alpha=0.9)
169 plt.plot(*args: time_true, ax_true, color='blue', label='Gerçek X İvmesi', alpha=0.7)
170 plt.xlabel('Zaman (saniye)')
171 plt.ylabel('İvme (m/s^2)')
172 plt.title('IMU X İvme Ölçümü vs. Gerçek X İvmesi')
173 plt.grid(True)
174 plt.legend()
175 plt.show()
176
177 plt.figure(figsize=(12, 6))
178 plt.plot(*args: time_true, imu_accel_y_meas, color='orange', label='IMU Y İvme Ölçümü', alpha=0.9)
179 plt.plot(*args: time_true, ay_true, color='blue', label='Gerçek Y İvmesi', alpha=0.7)
180 plt.xlabel('Zaman (saniye)')
181 plt.ylabel('İvme (m/s^2)')
182 plt.title('IMU Y İvme Ölçümü vs. Gerçek Y İvmesi')
183 plt.grid(True)
184 plt.legend()
185 plt.show()
186
187 # IMU Açısal Hız (Jiroskop) (Gerçek Açısal Hız ile Karşılaştırma)
188 plt.figure(figsize=(12, 6))
189 plt.plot(*args: time_true, np.degrees(imu_gyro_z_meas), color='orange', label='IMU Jiroskop Ölçümü (derece/s)',
190         alpha=0.9) # Radyandan dereceye çevir
191 plt.plot(*args: time_true, angular_velocity_true_deg, color='blue', label='Gerçek Açısal Hız (derece/s)', alpha=0.7)
192 plt.xlabel('Zaman (saniye)')
193 plt.ylabel('Açısal Hız (derece/s)')
194 plt.title('IMU Jiroskop Ölçümü vs. Gerçek Açısal Hız')
195 plt.grid(True)
196 plt.legend()
197 plt.show()
198
199 # GNSS Hız (Gerçek Hız ile Karşılaştırma)
200 # GNSS ölçümleri GNSS_UPDATE_INTERVAL'da bir yapıldığı için,
201 # GNSS verilerini çizerken zaman eksenini gnss_time_meas kullanmalıyız.
202 plt.figure(figsize=(12, 6))
203 gnss_speed_meas = np.sqrt(np.array(gnss_vx_meas) ** 2 + np.array(gnss_vy_meas) ** 2)
204 plt.plot(*args: gnss_time_meas, gnss_speed_meas, 'r.', label='GNSS Hız Ölçümleri (Gecikmeli ve Gürültülü)', alpha=0.5,
205         markersize=3)
206 plt.plot(*args: time_true, speed_true, color='blue', label='Gerçek Hız (m/s)', alpha=0.7)
207 plt.xlabel('Zaman (saniye)')
208 plt.ylabel('Hız (m/s)')
209 plt.title('GNSS Hız Ölçümleri vs. Gerçek Hız')
210 plt.grid(True)
211 plt.legend()
212 plt.show()
213
214 # Simüle edilmiş sensör verilerini kaydet
215 np.savez(file: 'sensor_data.npz',
216         imu_accel_x=np.array(imu_accel_x_meas),
217         imu_accel_y=np.array(imu_accel_y_meas),
218         imu_gyro_z=np.array(imu_gyro_z_meas),
219         gnss_x=np.array(gnss_x_meas),
220         gnss_y=np.array(gnss_y_meas),
221         gnss_vx=np.array(gnss_vx_meas),
222         gnss_vy=np.array(gnss_vy_meas),
223         gnss_time=np.array(gnss_time_meas),
224         gnss_delay_steps=GNSS_DELAY_STEPS, # Gecikme adım sayısı da kaydedilsin
225         dt=DT # Zaman adımı da kaydedilsin
226 )

```

DİPNOT:
KODDA ADIM-2 VE ADIM-3 YAPILMIŞTIR ANCAK ALTTAKİ
GRAFİKLER SADECE ADIM-2 NİN GRAFİKLERİDİR.
(ADIM 3 GEREKLİLİKLERİ YORUM SATIRIYLA BELİRTİLMİŞ AYNI
ZAMANDA ADIM 3 KISMINDA EKLENEN KODLAR GÖSTERİLMİŞ VE
ADIM 3 SONRASI GRAFİKLER GÜNCELLENMİŞTİR)





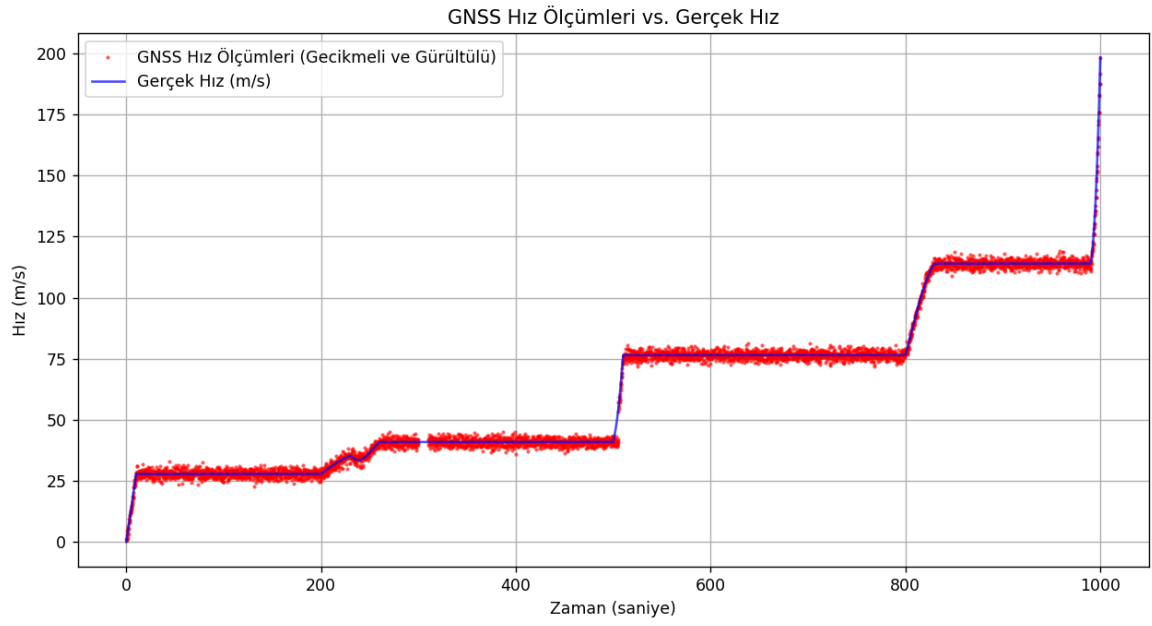
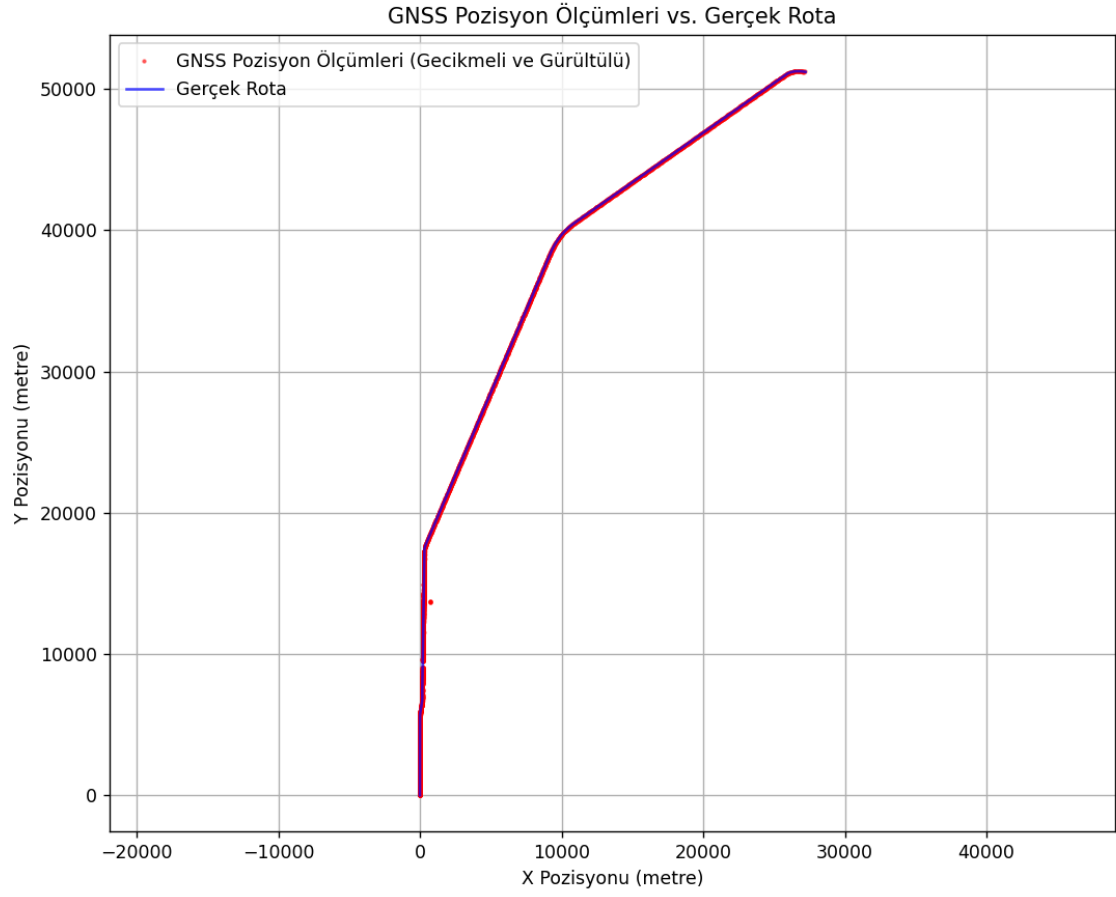


ADIM 3

GNSS SENSÖRÜNE VERİLEN ZAMANLARDAKİ BOZUCULARIN EKLENMESİ

```
59 # GNSS sabit veri üretme (flatline) durumu için global flag ve değerler
60 _gnss_flatline_active = False
61 _flatline_x_val = 0.0
62 _flatline_y_val = 0.0
63 _flatline_vx_val = 0.0
64 _flatline_vy_val = 0.0
```

```
rota.py  Sensor.py x
110 # - GNSS Sensörü Ek Bozuklukları (Adım 3 Gereklilikleri) -
111 current_time = time_true[i]
112
113 # 1. 300. ve 310. saniyeler arasında çıktı üretmeme
114 if 300.0 <= current_time <= 310.0:
115     continue # Bu adımda GNSS ölçümü yapılmaz, sonraki iterasyona geç
116
117 # 2. 400. saniyede 500 metre hatalı gösterme ve 1 saniye sonra düzeltme
118 if 400.0 <= current_time < (400.0 + 1.0):
119     delayed_x += 500.0
120     delayed_y += 500.0
121
122 # 3. 500. saniyede 5 saniye boyunca sabit veri üretme
123 if 500.0 <= current_time < 505.0:
124     if not _gnss_flatline_active:
125         # Sabitleme ilk kez başlıyor, o anki gecikmeli gerçek değeri kaydet
126         _flatline_x_val = delayed_x
127         _flatline_y_val = delayed_y
128         _flatline_vx_val = delayed_vx
129         _flatline_vy_val = delayed_vy
130         _gnss_flatline_active = True
131
132     # Sabitlenmiş değerleri kullan
133     delayed_x = _flatline_x_val
134     delayed_y = _flatline_y_val
135     delayed_vx = _flatline_vx_val
136     delayed_vy = _flatline_vy_val
137 else:
138     # Sabitleme süresi dışındaysa flag'i sıfırladık
139     _gnss_flatline_active = False
140
```



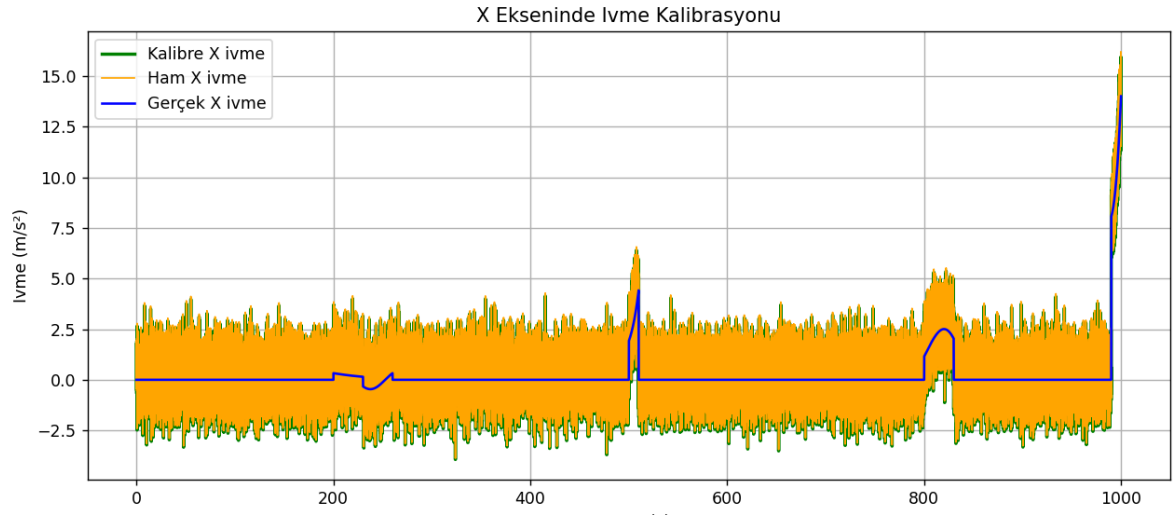
ADIM 4

SENSÖR KALİBRASYONU YAPILMASI

```
rota.py  Sensor.py  Calibration.py ×

1  import numpy as np
2  import matplotlib.pyplot as plt
3  #Verileri yükledim
4  true_data = np.load('true_state_data.npz')
5  sensor_data = np.load('sensor_data.npz')
6  time = true_data['time']
7  vx = true_data['vx']
8  vy = true_data['vy']
9  yaw = true_data['yaw']
10 acc_x_raw = sensor_data['imu_accel_x']
11 acc_y_raw = sensor_data['imu_accel_y']
12 gyro_raw = sensor_data['imu_gyro_z']
13 DT = sensor_data['dt']
14
15 #Altındaki değerleri sensor.py de kullanmıştık
16 ACCEL_BIAS_X = 0.1
17 ACCEL_BIAS_Y = 0.05
18 GYRO_BIAS = np.radians(0.1)
19
20 ACCEL_SCALE_X = 1.01
21 ACCEL_SCALE_Y = 0.99
22 GYRO_SCALE = 1.02
23
24 # - Kalibrasyon Formülü -
25 # (ölçüm / scale) - bias
26 acc_x_cal = (acc_x_raw / ACCEL_SCALE_X) - ACCEL_BIAS_X
27 acc_y_cal = (acc_y_raw / ACCEL_SCALE_Y) - ACCEL_BIAS_Y
28 gyro_cal = (gyro_raw / GYRO_SCALE) - GYRO_BIAS
29
30 # - Gerçek Değerleri Hesaplama (Karşılaştırma için) -
31 ax_true = np.diff(vx) / DT
32 ay_true = np.diff(vy) / DT
33 ax_true = np.append(ax_true, ax_true[-1])
34 ay_true = np.append(ay_true, ay_true[-1])
35 yaw_rad = np.radians(yaw)
36 yaw_rate_true = np.diff(yaw_rad) / DT
37 yaw_rate_true = np.append(yaw_rate_true, yaw_rate_true[-1])
```

```
39 # - Görselleştirme -
40 plt.figure(figsize=(12, 5))
41 plt.plot(*args: time, acc_x_cal, label="Kalibre X ivme", linewidth=2, color='green')
42 plt.plot(*args: time, acc_x_raw, label="Ham X ivme", color='orange', linewidth=1)
43 plt.plot(*args: time, ax_true, label="Gerçek X ivme", color='blue')
44 plt.xlabel("Zaman (s)")
45 plt.ylabel("İvme (m/s²)")
46 plt.title("X Ekseninde İvme Kalibrasyonu")
47 plt.grid(True)
48 plt.legend()
49
50 plt.figure(figsize=(12, 5))
51 plt.plot(*args: time, np.degrees(gyro_cal), label="Kalibre Jiroskop (°/s)", linewidth=2, color='green')
52 plt.plot(*args: time, np.degrees(gyro_raw), label="Ham Jiroskop (°/s)", color='orange', linewidth=1)
53 plt.plot(*args: time, np.degrees(yaw_rate_true), label="Gerçek Açılma Hız (°/s)", color='blue')
54 plt.xlabel("Zaman (s)")
55 plt.ylabel("Açılma Hız (°/s)")
56 plt.title("Jiroskop Kalibrasyonu")
57 plt.grid(True)
58 plt.legend()
59 plt.show()
60
61 # - Kalibre Edilmiş Veriyi Kaydet -
62 np.savez('file:calibrated_imu_data.npz',
63         acc_x=acc_x_cal,
64         acc_y=acc_y_cal,
65         gyro_z=gyro_cal,
66         dt=DT)
```

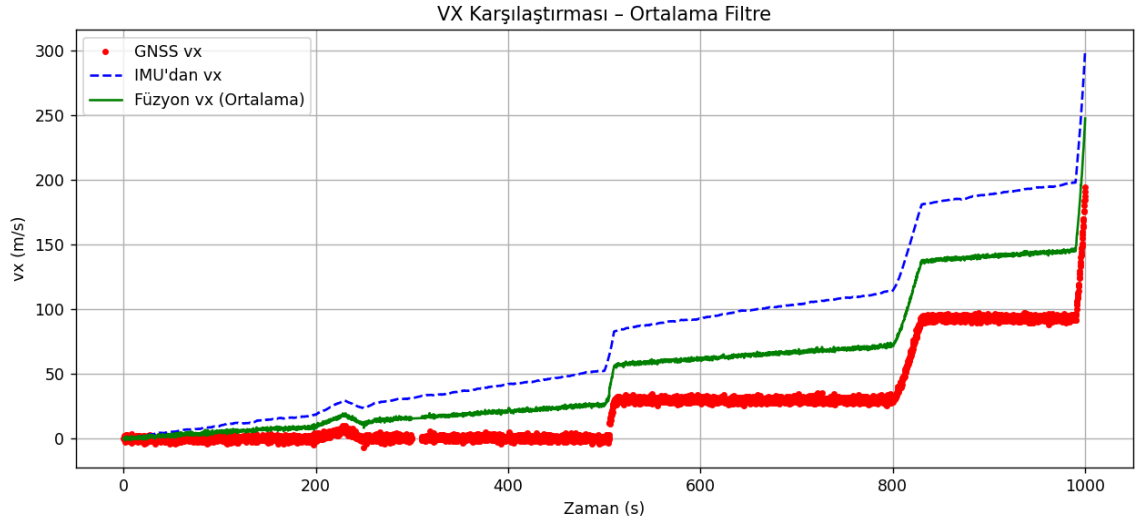
ADIM 5

FİLTRELEME TEKNİKLERİ İLE SENSÖR FÜZYONU YAPILMASI

İLK FİLTRE: SENSÖR ÇIKTILARINI TOPLAYIP İKİYE BÖLME (AVARAGE FİLTRE)

```
Filter_Avarage.py x rota.py Sensor.py Calibration.py
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Kalibre edilmiş IMU verilerini ve GNSS verilerini yükle
5 imu_data = np.load('calibrated_imu_data.npz')
6 sensor_data = np.load('sensor_data.npz')
7 acc_x = imu_data['acc_x']
8 acc_y = imu_data['acc_y']
9 dt = imu_data['dt']
10 gnss_vx = sensor_data['gnss_vx']
11 gnss_vy = sensor_data['gnss_vy']
12 gnss_t = sensor_data['gnss_time']
13
14 # IMU'dan hız hesaplamak (ivmeyi entegre et)
15 vx_imu = [0]
16 vy_imu = [0]
17
18 for i in range(1, len(acc_x)):
19     vx_imu.append(vx_imu[-1] + acc_x[i] * dt)
20     vy_imu.append(vy_imu[-1] + acc_y[i] * dt)
21
22 vx_imu = np.array(vx_imu)
23 vy_imu = np.array(vy_imu)
24
25 # GNSS verisi 5 Hz, IMU 50 Hz → GNSS zamanlarına karşılık gelen IMU hızlarını eşleştir
26 vx_imu_gnss = []
27 vy_imu_gnss = []
28
29 for t in gnss_t:
30     idx = int(t / dt)
31     vx_imu_gnss.append(vx_imu[idx])
32     vy_imu_gnss.append(vy_imu[idx])
33
34 vx_imu_gnss = np.array(vx_imu_gnss)
35 vy_imu_gnss = np.array(vy_imu_gnss)
```

```
37 # Basit füzyon: ortalama al
38 vx_avg = (vx_imu_gnss + gnss_vx) / 2
39 vy_avg = (vy_imu_gnss + gnss_vy) / 2
40
41 # Grafiklerle karşılaştır
42 plt.figure(figsize=(12, 5))
43 plt.plot(*args: gnss_t, gnss_vx, 'r.', label="GNSS vx")
44 plt.plot(*args: gnss_t, vx_imu_gnss, 'b--', label="IMU'dan vx")
45 plt.plot(*args: gnss_t, vx_avg, 'g-', label="Füzyon vx (Ortalama)")
46 plt.xlabel("Zaman (s)")
47 plt.ylabel("vx (m/s)")
48 plt.title("VX Karşılaştırması - Ortalama Filtre")
49 plt.grid(True)
50 plt.legend()
51 plt.show()
```



İKİNCİ FİLTRE : ALÇAK GEÇİREN FİLTRE (LOWPASS FİLTRE)

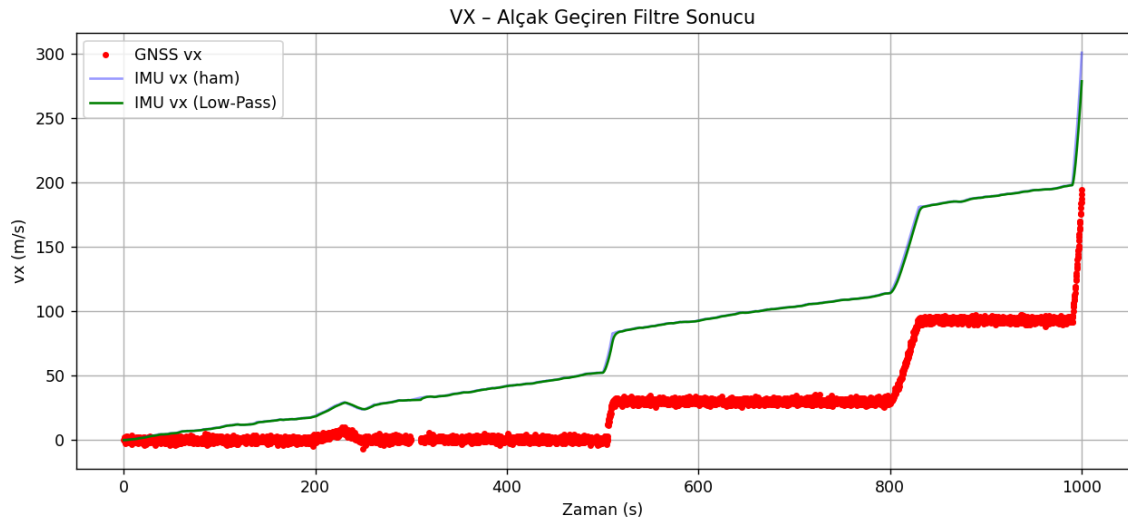
```
Filter_Avarage.py  Filter_Lowpass.py  ×  rota.py  Sensor.py  Calibration.py

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  # Kalibre edilmiş IMU verisi
5  imu_data = np.load('calibrated_imu_data.npz')
6  sensor_data = np.load('sensor_data.npz')
7
8  acc_x = imu_data['acc_x']
9  acc_y = imu_data['acc_y']
10 dt = imu_data['dt']
11
12 gnss_vx = sensor_data['gnss_vx']
13 gnss_vy = sensor_data['gnss_vy']
14 gnss_t = sensor_data['gnss_time']
15
16 # IMU ivmesini entegre ederek hız hesapla
17 vx = [0]
18 vy = [0]
19 for i in range(1, len(acc_x)):
20     vx.append(vx[-1] + acc_x[i] * dt)
21     vy.append(vy[-1] + acc_y[i] * dt)
22
23 vx = np.array(vx)
24 vy = np.array(vy)
25
26 # GNSS zamanlarına karşılık gelen IMU hızları
27 vx_imu = []
28 vy_imu = []
29
30 for t in gnss_t:
31     idx = int(t / dt)
32     vx_imu.append(vx[idx])
33     vy_imu.append(vy[idx])
34 vx_imu = np.array(vx_imu)
35 vy_imu = np.array(vy_imu)
```

```

37 # Alçak geçiren filtre uygulanacak
38 def low_pass_filter(data, alpha=0.9): 2 usages
39     filtered = [data[0]]
40     for n in range(1, len(data)):
41         filts = alpha * filtered[-1] + (1 - alpha) * data[n]
42         filtered.append(filts)
43     return np.array(filtered)
44 # Uygulama
45 vx_lpf = low_pass_filter(vx_imu, alpha=0.9)
46 vy_lpf = low_pass_filter(vy_imu, alpha=0.9)
47
48 # Karşılaştırma Grafiği
49 plt.figure(figsize=(12, 5))
50 plt.plot(*args: gnss_t, gnss_vx, 'r.', label="GNSS vx")
51 plt.plot(*args: gnss_t, vx_imu, 'blue', alpha=0.4, label="IMU vx (ham)")
52 plt.plot(*args: gnss_t, vx_lpf, 'g-', label="IMU vx (Low-Pass)")
53 plt.xlabel("Zaman (s)")
54 plt.ylabel("vx (m/s)")
55 plt.title("VX - Alçak Geçiren Filtre Sonucu")
56 plt.grid(True)
57 plt.legend()
58 plt.show()

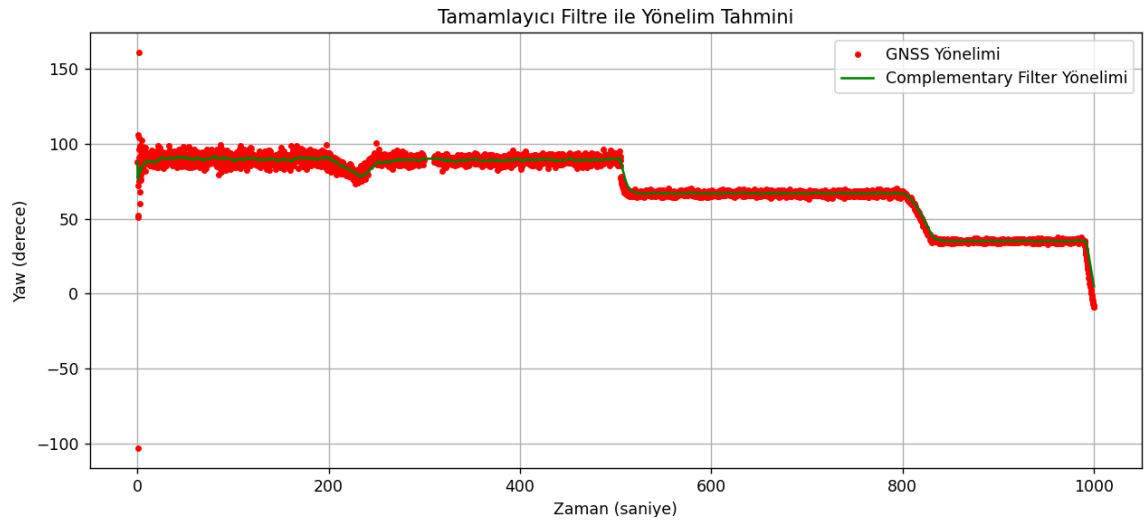
```



ÜÇÜNCÜ FİLTRE : TAMAMLAYICI (COMPLEMENTARY) FİLTRE

```
Filter_Complementary.py x Filter_Avarage.py Filter_Lowpass.py rota.py Sensor.py C

1 import numpy as np
2 import matplotlib.pyplot as plt
3 # Kalibre edilmiş IMU jiroskop verisini yükle
4 imu_data = np.load('calibrated_imu_data.npz')
5 gyro_z = imu_data['gyro_z']
6 dt = imu_data['dt']
7 # GNSS verileri
8 sensor_data = np.load('sensor_data.npz')
9 gnss_vx = sensor_data['gnss_vx']
10 gnss_vy = sensor_data['gnss_vy']
11 gnss_t = sensor_data['gnss_time']
12
13 # Complementary filter parametresi
14 alpha = 0.95
15
16 # Başlangıç yönelimi
17 yaw_est = [np.arctan2(gnss_vy[0], gnss_vx[0])]
18
19 for i in range(1, len(gnss_t)):
20     # GNSS yönelimi
21     yaw_gnss = np.arctan2(gnss_vy[i], gnss_vx[i])
22
23     # IMU ile tahmin (gyro_z GNSS zamanına denk geldiği varsayılıyor)
24     yaw_imu = yaw_est[-1] + gyro_z[int(gnss_t[i] / dt)] * dt
25
26     # Complementary filtre
27     yaw_fused = alpha * yaw_imu + (1 - alpha) * yaw_gnss
28
29     # Açığı -pi ile pi arasına getir
30     yaw_fused = (yaw_fused + np.pi) % (2 * np.pi) - np.pi
31     yaw_est.append(yaw_fused)
32
33 # Sonucu dereceye çevir
34 yaw_est_deg = np.degrees(yaw_est)
35 yaw_gnss_deg = np.degrees(np.arctan2(gnss_vy, gnss_vx))
36
37 # Kaydet
38 np.savez('complementary_filter.npz',
39         yaw_deg=yaw_est_deg,
40         time=gnss_t)
41 # Görselleştirme
42 plt.figure(figsize=(12, 5))
43 plt.plot(*args: gnss_t, yaw_gnss_deg, 'r.', label="GNSS Yönelimi")
44 plt.plot(*args: gnss_t, yaw_est_deg, 'g-', label="Complementary Filter Yönelimi")
45 plt.xlabel("Zaman (saniye)")
46 plt.ylabel("Yaw (derece)")
47 plt.title("Tamamlayıcı Filtre ile Yönelim Tahmini")
48 plt.grid(True)
49 plt.legend()
50 plt.show()
```



DÖRDÜNCÜ FİLTRE:EKF

Sistemin nonlinear olduğu için ekf kullanıcağım. Sistemimin nonlinear olduğunu kanıtlayan kodlar :

#Trigonometrik fonksiyonlar kullanıldı (cos,sin)

$ax = avg_accel_magnitude * \cos(yaw)$

$ay = avg_accel_magnitude * \sin(yaw)$

vx ve vy güncellenirken, değişken (yaw'a bağlı) ivme değerleriyle çarpım yapılıyor

$vx += ax * DT$

$vy += ay * DT$

```
Filter_EKF.py x Filter_Complementary.py Filter_Avarage.py Filter_Lowpass.py rota.py
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def f(x, u, dt): 1 usage
5     x_pos, y_pos, v, yaw = x
6     a, omega = u
7     x_pos += v * np.cos(yaw) * dt
8     y_pos += v * np.sin(yaw) * dt
9     v += a * dt
10    yaw += omega * dt
11    return np.array([x_pos, y_pos, v, yaw])
12
13 def jacobian_f(x, u, dt): 1 usage
14     _, _, v, yaw = x
15     F = np.eye(4)
16     F[0, 2] = np.cos(yaw) * dt
17     F[0, 3] = -v * np.sin(yaw) * dt
18     F[1, 2] = np.sin(yaw) * dt
19     F[1, 3] = v * np.cos(yaw) * dt
20     return F
21
22 def h(x): 1 usage
23     x_pos, y_pos, v, yaw = x
24     vx = v * np.cos(yaw)
25     vy = v * np.sin(yaw)
26     return np.array([x_pos, y_pos, vx, vy])
27
28 def jacobian_h(x): 1 usage
29     _, _, v, yaw = x
30     H = np.zeros((4, 4))
31     H[0, 0] = 1
32     H[1, 1] = 1
33     H[2, 2] = np.cos(yaw)
34     H[2, 3] = -v * np.sin(yaw)
35     H[3, 2] = np.sin(yaw)
36     H[3, 3] = v * np.cos(yaw)
37     return H
```

```

39 def ekf_predict(x, P, u, dt, Q): 1 usage
40     F = jacobian_f(x, u, dt)
41     x_pred = f(x, u, dt)
42     P_pred = F @ P @ F.T + Q
43     P_pred = np.clip(P_pred, a_min=None, a_max=1e3)
44     return x_pred, P_pred
45
46 def ekf_update(x_pred, P_pred, z, R): 1 usage
47     H = jacobian_h(x_pred)
48     y = z - h(x_pred)
49     S = H @ P_pred @ H.T + R
50     K = P_pred @ H.T @ np.linalg.inv(S)
51     x_updated = x_pred + K @ y
52     P_updated = (np.eye(len(x_pred)) - K @ H) @ P_pred
53     return x_updated, P_updated
54
55 # Veri Yükleme
56 imu_data = np.load('calibrated_imu_data.npz')
57 sensor_data = np.load('sensor_data.npz')
58
59 acc_x = imu_data['acc_x']
60 acc_y = imu_data['acc_y']
61 gyro_z = imu_data['gyro_z']
62 dt = imu_data['dt'].item()
63
64 gnss_x = sensor_data['gnss_x']
65 gnss_y = sensor_data['gnss_y']
66 gnss_vx = sensor_data['gnss_vx']
67 gnss_vy = sensor_data['gnss_vy']
68 gnss_time = sensor_data['gnss_time']
69

```

```

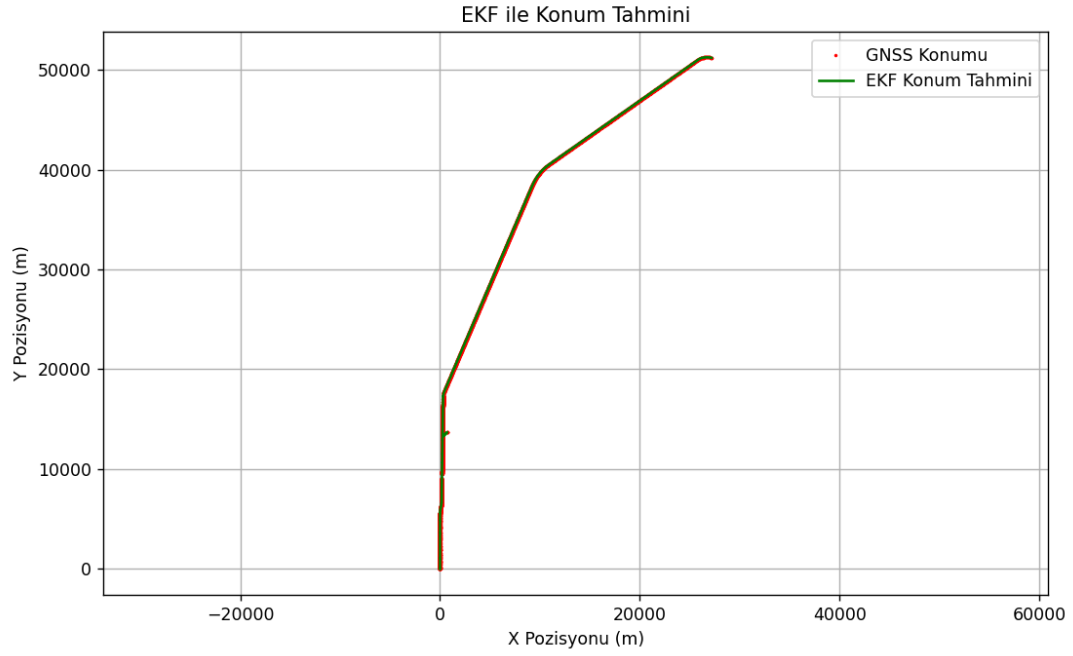
70 #EKF Döngüsü
71 x = np.array([gnss_x[0], gnss_y[0], 0.0, np.deg2rad(90)])
72 P = np.eye(4) * 1.0
73 Q = np.diag([0.1, 0.1, 0.3, np.radians(2)])**2
74 R = np.diag([3.0, 3.0, 0.5, 0.5])**2
75 x_estimates = []
76 time_estimates = []
77 P_all = []
78 gnss_index = 0
79
80 for i in range(len(acc_x)):
81     a_forward = acc_x[i] * np.cos(x[3]) + acc_y[i] * np.sin(x[3])
82     u = np.array([a_forward, gyro_z[i]])
83     # Prediction
84     x, P = ekf_predict(x, P, u, dt, Q)
85     # GNSS varsı update
86     t = i * dt
87     if gnss_index < len(gnss_time) and np.isclose(t, gnss_time[gnss_index], atol=dt/2):
88         z = np.array([gnss_x[gnss_index], gnss_y[gnss_index], gnss_vx[gnss_index], gnss_vy[gnss_index]])
89         x, P = ekf_update(x, P, z, R)
90         gnss_index += 1
91         x_estimates.append(x.copy())
92         time_estimates.append(t)
93         P_all.append(P.copy())
94
95 x_estimates = np.array(x_estimates)
96 time_estimates = np.array(time_estimates)
97 P_all_array = np.stack(P_all, axis=0)
98
99 #Kaydet (Visualization.py için)
100 np.savez( file: "ekf_estimates.npz",
101          x_estimates=x_estimates,
102          time=time_estimates,
103          P_all=P_all_array)
104

```

```

105 # Görselleştirme
106 plt.figure(figsize=(10, 6))
107 plt.plot(*args: gnss_x, gnss_y, 'r.', alpha=0.4, markersize=3, label='GNSS Konumu')
108 plt.plot(*args: x_estimates[:, 0], x_estimates[:, 1], 'g-', linewidth=2, label='EKF Konum Tahmini')
109 plt.xlabel("X Pozisyonu (m)")
110 plt.ylabel("Y Pozisyonu (m)")
111 plt.title("EKF ile Konum Tahmini")
112 plt.axis('equal')
113 plt.grid(True)
114 plt.legend()
115 plt.show()

```



ADIM 6

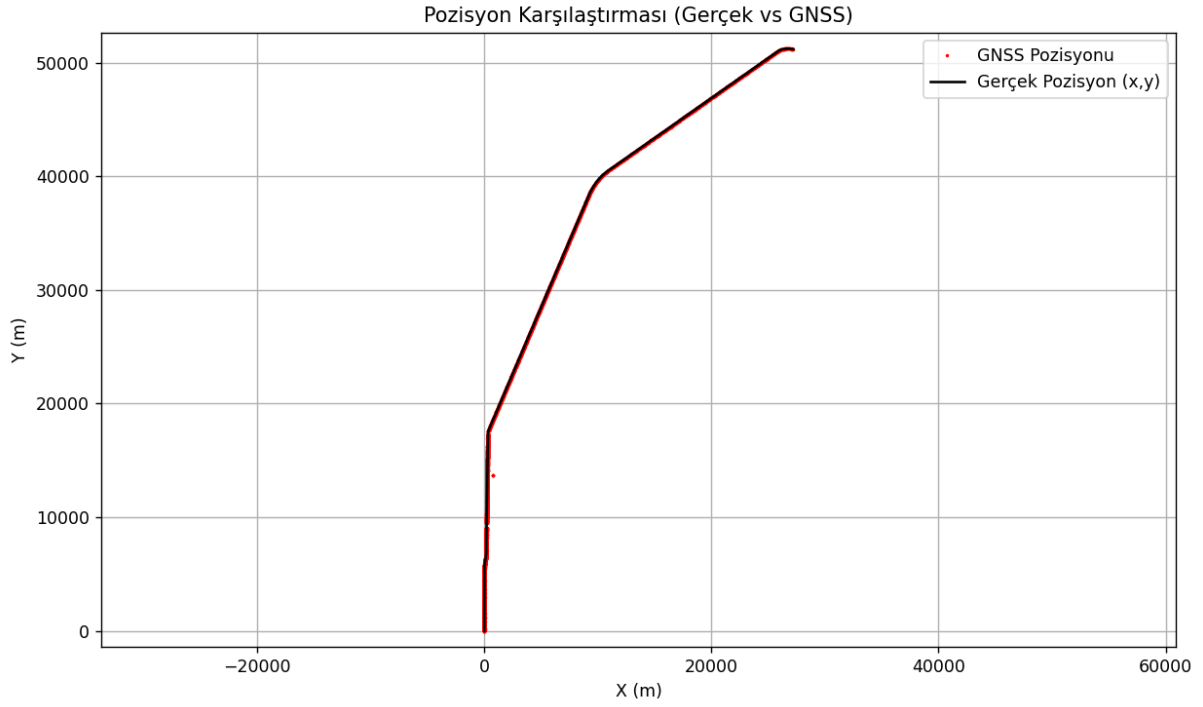
ÇIKTILARI GRAFİKLER ÜZERİNDEN ÇİZİLMESİ

Gerekli grafikleri bu adımda tekrar ya da yeni çizerek istenen grafikleri tek bir yerde topladım

Gerekli grafikler için gereken verileri yükledim;

```
Visualization.py × Filter_EKF.py Filter_Complementary.py Filter_Avarage.py Filter_Lowpass.p
1 import numpy as np
2 import matplotlib.pyplot as plt
3 # - Veri Yükleme -|
4 true = np.load("true_state_data.npz")
5 sensor = np.load("sensor_data.npz")
6 imu = np.load("calibrated_imu_data.npz")
7 ekf_data = np.load("ekf_estimates.npz")
8 complementary_data = np.load("complementary_filter.npz")
9
10 # Temel veriler
11 time = true['time']
12 x_true = true['x']
13 y_true = true['y']
14 ax_true = np.diff(true['vx']) / sensor['dt'][(0)]
15 ay_true = np.diff(true['vy']) / sensor['dt'][(0)]
16 ax_true = np.append(ax_true, ax_true[-1])
17 ay_true = np.append(ay_true, ay_true[-1])
18
19 acc_x_raw = sensor['imu_accel_x']
20 acc_y_raw = sensor['imu_accel_y']
21 acc_x_cal = imu['acc_x']
22 acc_y_cal = imu['acc_y']
23
24 gyro_raw = sensor['imu_gyro_z']
25 gyro_cal = imu['gyro_z']
26 yaw_true = true['yaw']
27 yaw_rad_unwrapped = np.unwrap(np.radians(yaw_true)) # Ani sıçramaları önlemek için düzleştirme yapıldı
28 yaw_rate_true = np.diff(yaw_rad_unwrapped) / sensor['dt'][(0)]
29 yaw_rate_true = np.append(yaw_rate_true, yaw_rate_true[-1])
30
31
32 gnss_x = sensor['gnss_x']
33 gnss_y = sensor['gnss_y']
34 gnss_time = sensor['gnss_time']
35 x_estimates = ekf_data['x_estimates']
36 time_est = ekf_data['time']
37 P_all = ekf_data['P_all']
```

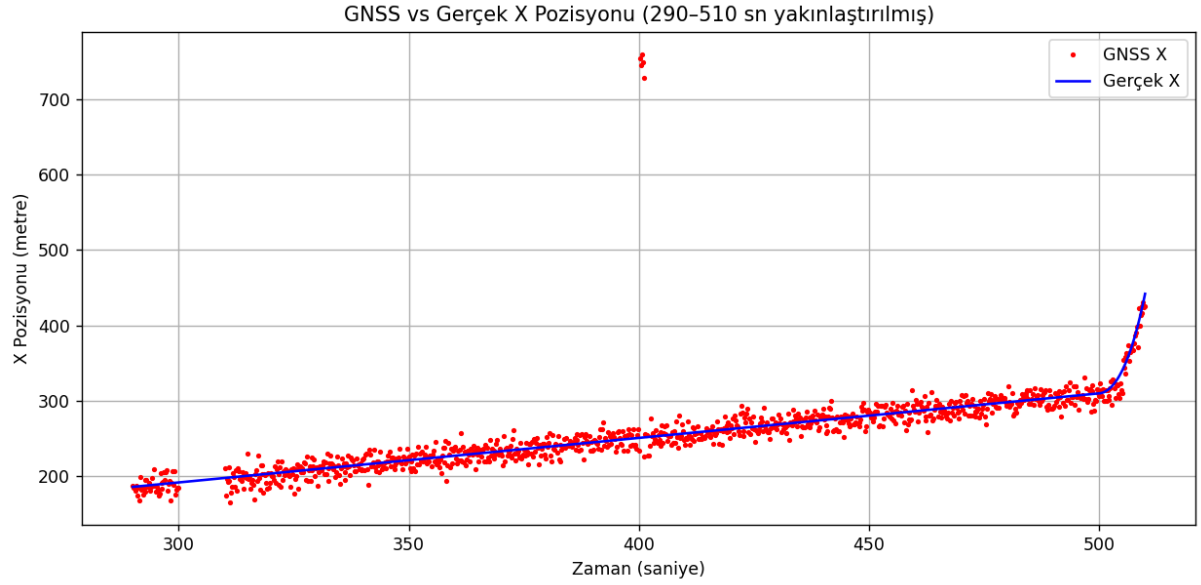
```
39 # - 1. Pozisyon Grafiği -
40 plt.figure(figsize=(10, 6))
41 plt.plot(*args: gnss_x, gnss_y, 'r.', markersize=2, label="GNSS Pozisyonu")
42 plt.plot(*args: x_true, y_true, label="Gerçek Pozisyon (x,y)", color='black')
43 plt.xlabel("X (m)")
44 plt.ylabel("Y (m)")
45 plt.title("Pozisyon Karşılaştırması (Gerçek vs GNSS)")
46 plt.grid(True)
47 plt.axis("equal")
48 plt.legend()
49 plt.tight_layout()
50 plt.show()
```



```

52 # - 2. GNSS X Pozisyonu Zamana Karşı Değişimi -
53 mask_gnss = (gnss_time >= 290) & (gnss_time <= 510)
54 mask_true = (time >= 290) & (time <= 510)
55
56 plt.figure(figsize=(10, 5))
57 plt.plot(*args: gnss_time[mask_gnss], gnss_x[mask_gnss], 'r.', label='GNSS X', markersize=4)
58 plt.plot(*args: time[mask_true], x_true[mask_true], 'b-', label='Gerçek X')
59 plt.xlabel("Zaman (saniye)")
60 plt.ylabel("X Pozisyonu (metre)")
61 plt.title("GNSS vs Gerçek X Pozisyonu (290-510 sn yakınlaştırılmış)")
62 plt.grid(True)
63 plt.legend()
64 plt.tight_layout()
65 plt.show()

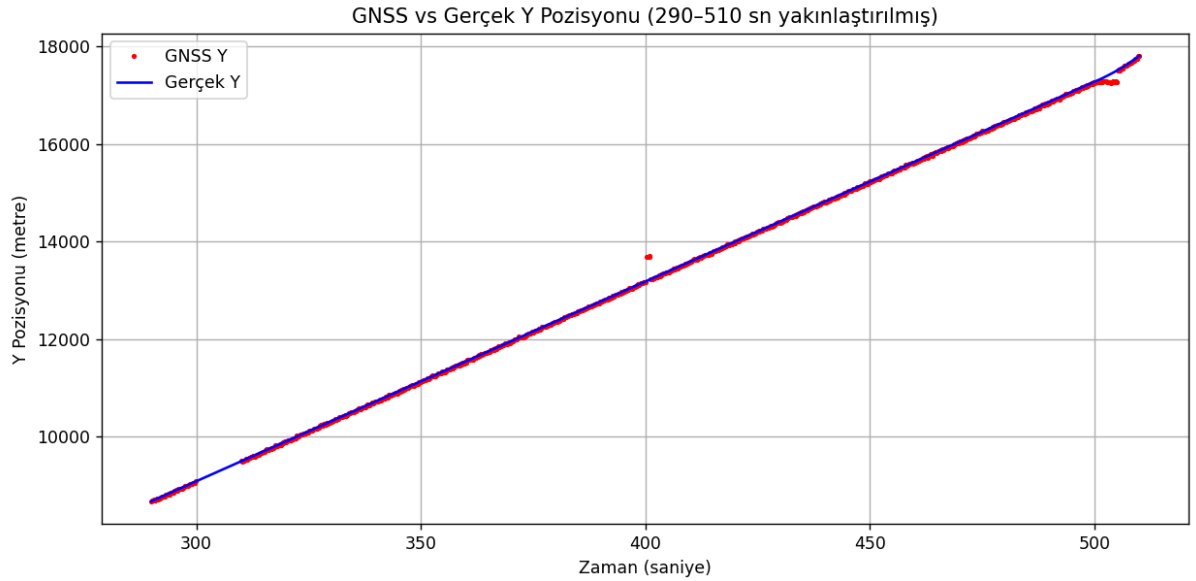
```



```

67 # - 3. GNSS Y Pozisyonu Zamana Karşı Değişimi -
68 plt.figure(figsize=(10, 5))
69 plt.plot(*args: gnss_time[mask_gnss], gnss_y[mask_gnss], 'r.', label='GNSS Y', markersize=4)
70 plt.plot(*args: time[mask_true], y_true[mask_true], 'b-', label='Gerçek Y')
71 plt.xlabel("Zaman (saniye)")
72 plt.ylabel("Y Pozisyonu (metre)")
73 plt.title("GNSS vs Gerçek Y Pozisyonu (290-510 sn yakınlaştırılmış)")
74 plt.grid(True)
75 plt.legend()
76 plt.tight_layout()
77 plt.show()

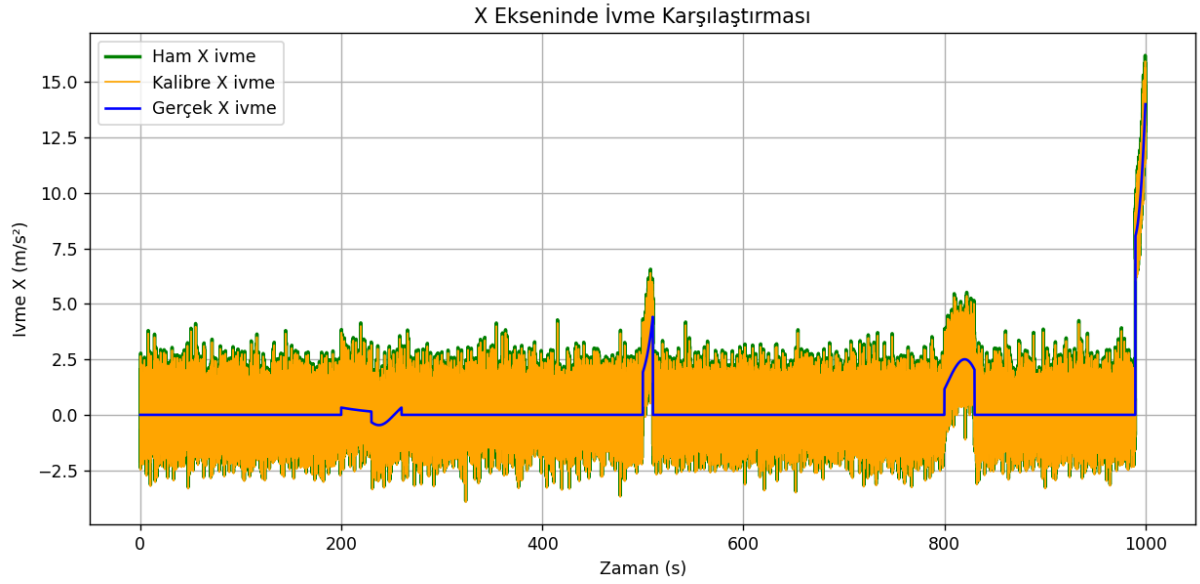
```



```

79 # - 4. X Ekseninde İvme Kalibrasyonu -
80 plt.figure(figsize=(10, 5))
81 plt.plot(*args: time, acc_x_raw, color='green',linewidth=2, label='Ham X ivme')
82 plt.plot(*args: time, acc_x_cal, color='orange',linewidth=1, label='Kalibre X ivme')
83 plt.plot(*args: time, ax_true, color='blue',label='Gerçek X ivme')
84
85 plt.xlabel("Zaman (s)")
86 plt.ylabel("İvme X (m/s²)")
87 plt.title("X Ekseninde İvme Karşılaştırması")
88 plt.grid(True)
89 plt.legend()
90 plt.tight_layout()
91 plt.show()

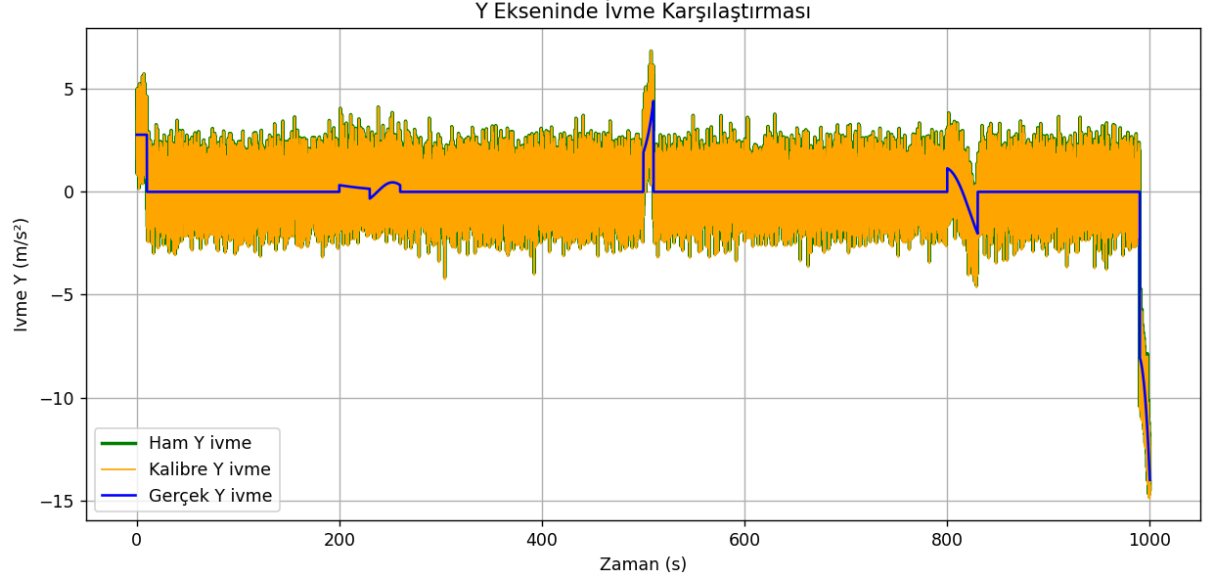
```



```

93 # - 5. Y Ekseninde İvme Kalibrasyonu -
94 plt.figure(figsize=(10, 5))
95 plt.plot(*args: time, acc_y_raw,color='green',linewidth=2, label="Ham Y ivme")
96 plt.plot(*args: time, acc_y_cal,color='orange',linewidth=1, label="Kalibre Y ivme" )
97 plt.plot(*args: time, ay_true, label="Gerçek Y ivme", color='blue')
98 plt.xlabel("Zaman (s)")
99 plt.ylabel("İvme Y (m/s²)")
100 plt.title("Y Ekseninde İvme Karşılaştırması")
101 plt.grid(True)
102 plt.legend()
103 plt.tight_layout()
104 plt.show()
105

```



```

106 # - 6. Açısal Hız Karşılaştırması -
107 plt.figure(figsize=(10, 5))
108 plt.plot(*args: time, np.degrees(gyro_raw), label="Ham Jiroskop (°/s)", color='green',linewidth=2)
109 plt.plot(*args: time, np.degrees(gyro_cal), label="Kalibre Jiroskop (°/s)",color='orange',linewidth=1)
110 plt.plot(*args: time, np.degrees(yaw_rate_true), label="Gerçek Açısal Hız (°/s)", color='blue')
111 plt.xlabel("Zaman (s)")
112 plt.ylabel("Açısal Hız (°/s)")
113 plt.title("Açısal Hız (Yaw Rate) Karşılaştırması")
114 plt.grid(True)
115 plt.legend()
116 plt.tight_layout()
117 plt.show()
118

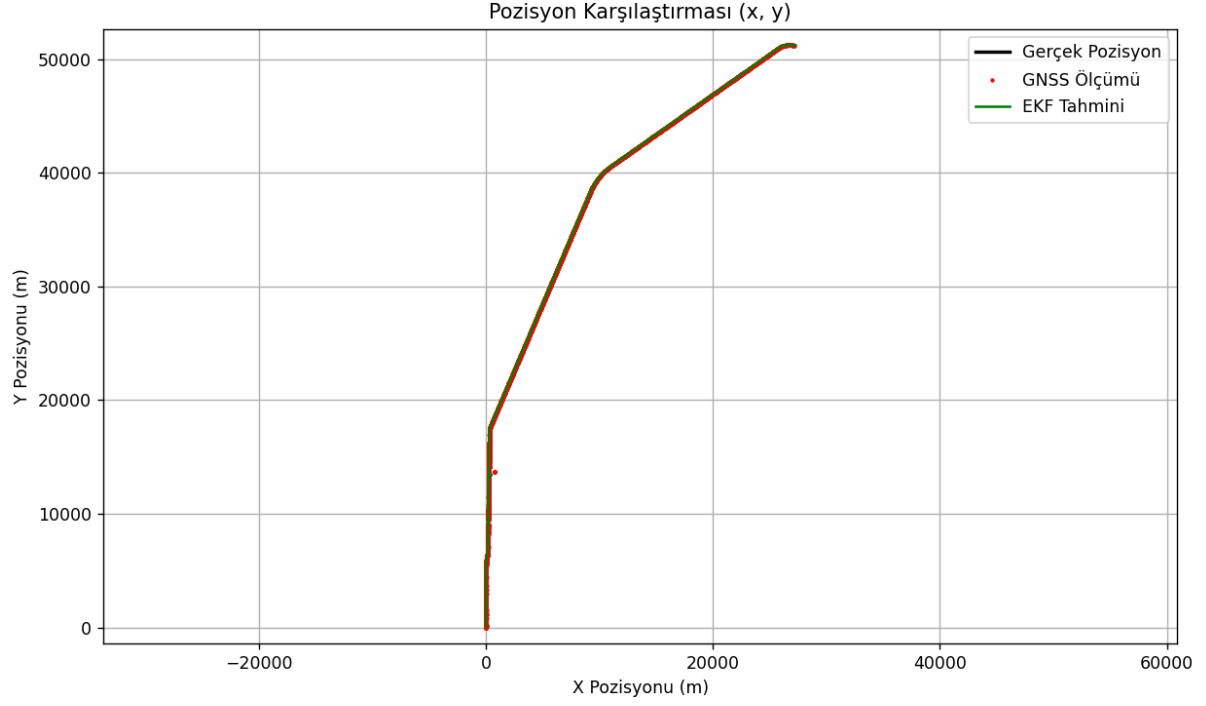
```



```

119 # - 7. Pozisyon Karşılaştırması (x, y) -
120 plt.figure(figsize=(10, 6))
121 plt.plot(*args: x_true, y_true, label="Gerçek Pozisyon", color='black', linewidth=2)
122 plt.plot(*args: gnss_x, gnss_y, 'r.', markersize=3, label='GNSS Ölçümü')
123 plt.plot(*args: x_estimates[:, 0], x_estimates[:, 1], 'g-', label='EKF Tahmini')
124 plt.xlabel("X Pozisyonu (m)")
125 plt.ylabel("Y Pozisyonu (m)")
126 plt.title("Pozisyon Karşılaştırması (x, y)")
127 plt.axis('equal')
128 plt.grid(True)
129 plt.legend()
130 plt.tight_layout()
131 plt.show()

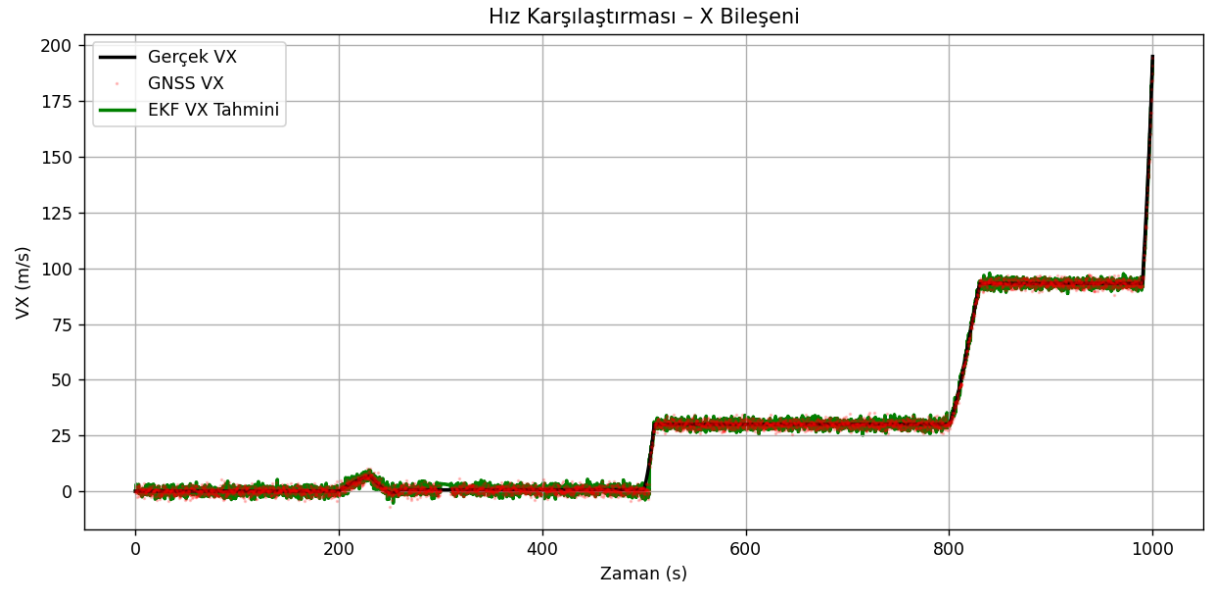
```



```

133 # - 8. Hız Karşılaştırması (VX) -
134 vx_true = true['vx']
135 vx_gnss = sensor['gnss_vx']
136 vx_ekf = x_estimates[:, 2] * np.cos(x_estimates[:, 3])
137 plt.figure(figsize=(10, 5))
138 plt.plot(*args: time, vx_true, label='Gerçek VX', color='black',linewidth=2)
139 plt.plot(*args: gnss_time, vx_gnss, 'r.',alpha=0.2, markersize=2, label='GNSS VX')
140 plt.plot(*args: time_est, vx_ekf, label='EKF VX Tahmini',linewidth=2, color='green',zorder=1)
141 plt.xlabel("Zaman (s)")
142 plt.ylabel("VX (m/s)")
143 plt.title("Hız Karşılaştırması - X Bileşeni")
144 plt.grid(True)
145 plt.legend()
146 plt.tight_layout()
147 plt.show()

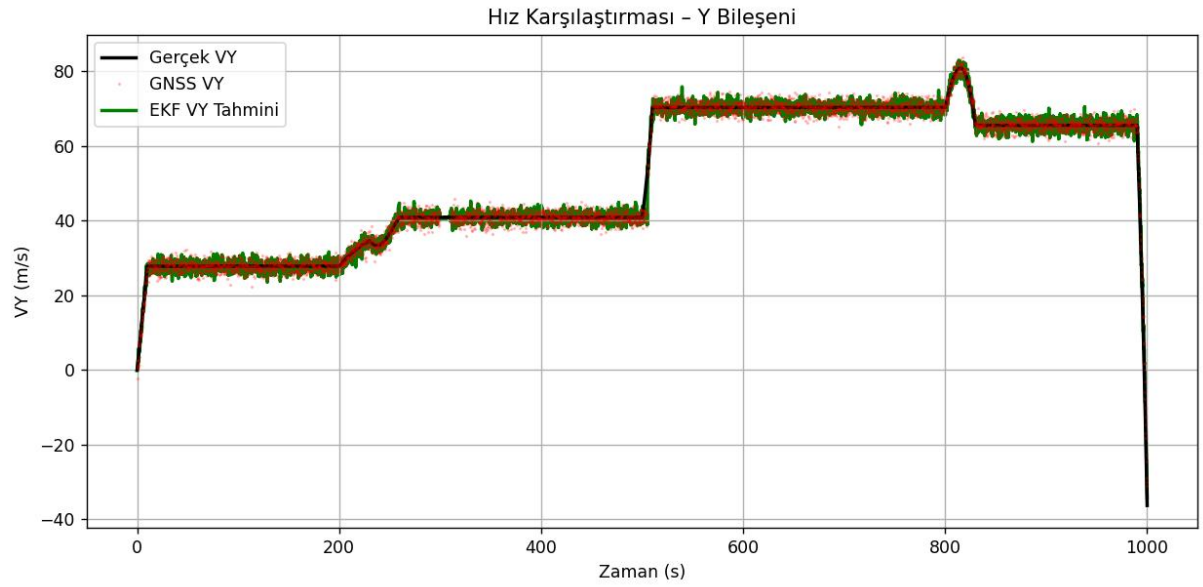
```



```

149 # - 9. Hız Karşılaştırması (VY) -
150 vy_true = true['vy']
151 vy_gnss = sensor['gnss_vy']
152 vy_ekf = x_estimates[:, 2] * np.sin(x_estimates[:, 3])
153 plt.figure(figsize=(10, 5))
154 plt.plot(*args: time, vy_true, label='Gerçek VY', color='black',linewidth=2)
155 plt.plot(*args: gnss_time, vy_gnss, 'r.',alpha=0.2, markersize=2, label='GNSS VY')
156 plt.plot(*args: time_est, vy_ekf, label='EKF VY Tahmini',linewidth=2, color='green',zorder=1)
157 plt.xlabel("Zaman (s)")
158 plt.ylabel("VY (m/s)")
159 plt.title("Hız Karşılaştırması - Y Bileşeni")
160 plt.grid(True)
161 plt.legend()
162 plt.tight_layout()
163 plt.show()
164

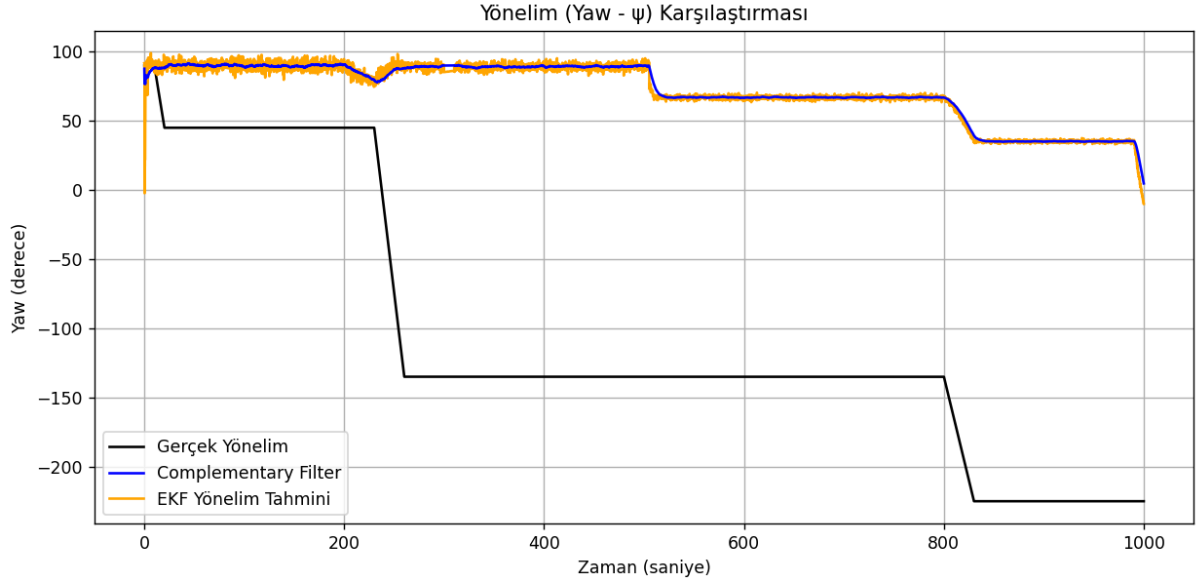
```




```

165 # - 10. Yönelim Karşılaştırması -
166 yaw_true_deg = true['yaw']
167 yaw_complementary_deg = complementary_data['yaw_deg']
168 complementary_time = complementary_data['time']
169 yaw_ekf_deg = np.degrees(x_estimates[:, 3])
170 plt.figure(figsize=(10, 5))
171 plt.plot(*args: time, yaw_true_deg, label="Gerçek Yönelim", color='black',zorder=2)
172 plt.plot(*args: complementary_time, yaw_complementary_deg, label="Complementary Filter", color='blue',zorder=3)
173 plt.plot(*args: time_est, yaw_ekf_deg, label="EKF Yönelim Tahmini", color='orange')
174 plt.xlabel("Zaman (saniye)")
175 plt.ylabel("Yaw (derece)")
176 plt.title("Yönelim (Yaw -  $\psi$ ) Karşılaştırması")
177 plt.grid(True)
178 plt.legend()
179 plt.tight_layout()
180 plt.show()

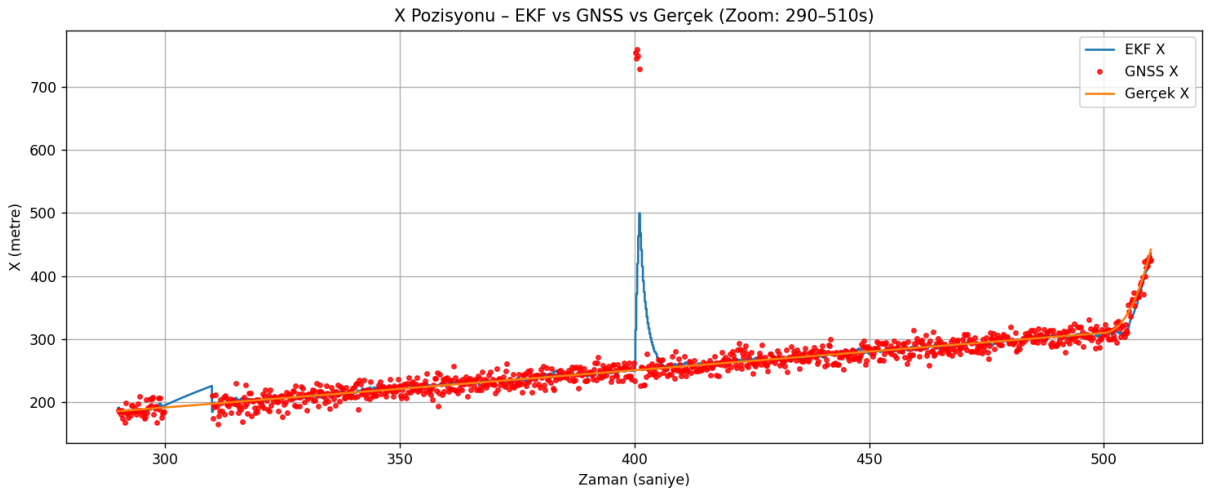
```



```

182 # - 11. EKF vs GNSS vs Gerçek (Zoom: 290-510s) -
183 mask_zoom_true = (time >= 290) & (time <= 510)
184 mask_zoom_gnss = (gnss_time >= 290) & (gnss_time <= 510)
185 mask_zoom_est = (time_est >= 290) & (time_est <= 510)
186 plt.figure(figsize=(12, 5))
187 plt.plot(*args: time_est[mask_zoom_est], x_estimates[mask_zoom_est, 0], label='EKF X')
188 plt.plot(*args: gnss_time[mask_zoom_gnss], gnss_x[mask_zoom_gnss], 'r.',alpha=0.8, label='GNSS X')
189 plt.plot(*args: time[mask_zoom_true], x_true[mask_zoom_true], label='Gerçek X')
190 plt.title("X Pozisyonu - EKF vs GNSS vs Gerçek (Zoom: 290-510s)")
191 plt.xlabel("Zaman (saniye)")
192 plt.ylabel("X (metre)")
193 plt.grid(True)
194 plt.legend()
195 plt.tight_layout()
196 plt.show()

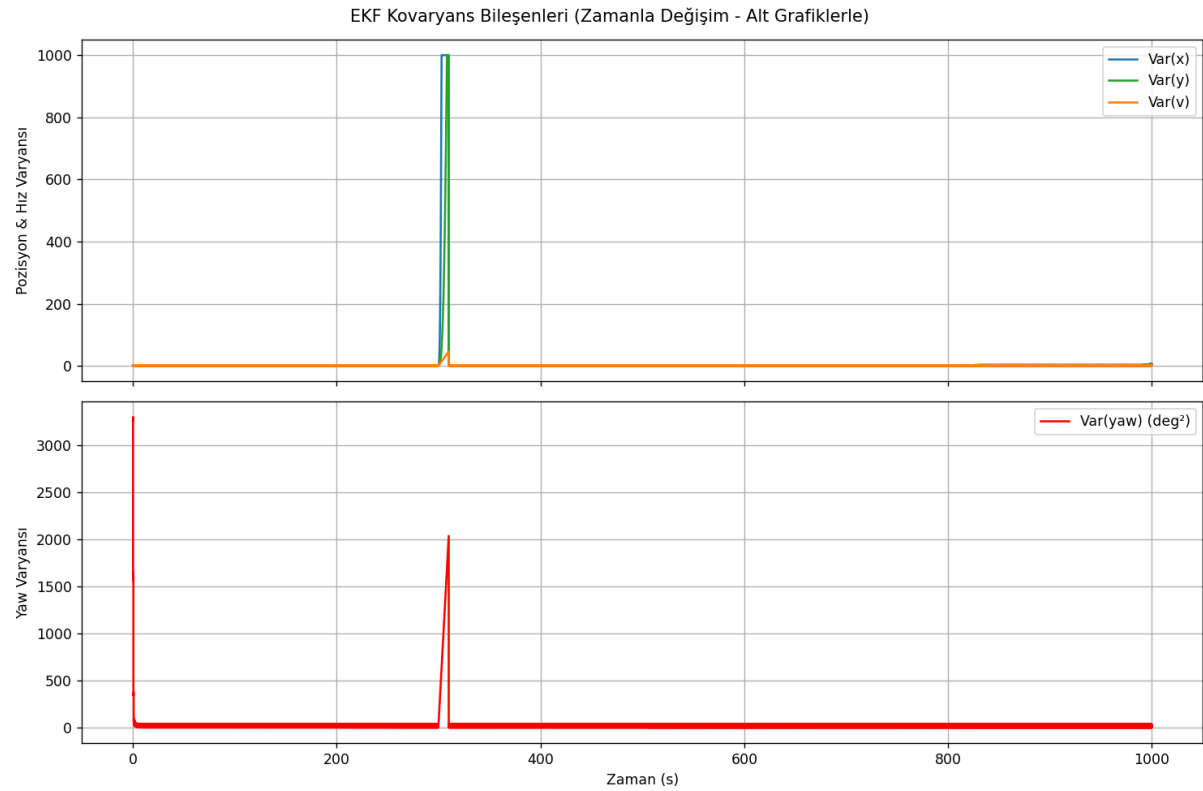
```



```

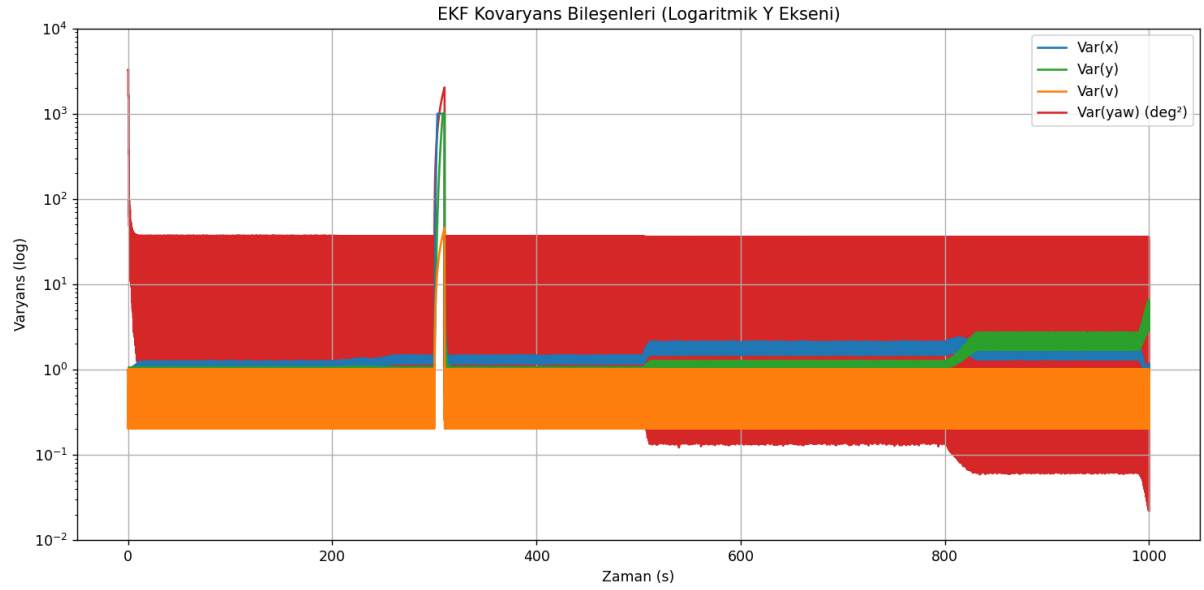
198 # - 12.1 EKF Kovaryans Matrisinin Zamanla Değişimi (Alt Alta Ayrı Grafiklerle) -
199 # Radyandan dereceye dönüşüm için katsayı
200 rad2deg_squared = (180 / np.pi) ** 2
201 fig, axs = plt.subplots(nrows=2, ncols=1, figsize=(12, 8), sharex=True)
202 # Pozisyon ve hız varyansları
203 axs[0].plot(time_est, P_all[:, 0, 0], label="Var(x)", color='#1f77b4')
204 axs[0].plot(time_est, P_all[:, 1, 1], label="Var(y)", color='#2ca02c')
205 axs[0].plot(time_est, P_all[:, 2, 2], label="Var(v)", color='#ff7f0e')
206 axs[0].set_ylabel("Pozisyon & Hız Varyansı")
207 axs[0].legend()
208 axs[0].grid(True)
209 # Yaw varyansı
210 axs[1].plot(time_est, P_all[:, 3, 3] * rad2deg_squared, label="Var(yaw) (deg²)", color='red')
211 axs[1].set_xlabel("Zaman (s)")
212 axs[1].set_ylabel("Yaw Varyansı")
213 axs[1].legend()
214 axs[1].grid(True)
215 plt.suptitle("EKF Kovaryans Bileşenleri (Zamanla Değişim - Alt Grafiklerle)")
216 plt.tight_layout()
217 plt.show()

```



DAHA RAHAT ANLAŞILMASI İÇİN LOGARİTMİK GRAFİK ÇİZDİRDİM

```
219 # - 12.2 EKF Kovaryans Matrisinin Logaritmik Y Eksenine Çizimi -
220 plt.figure(figsize=(12, 6))
221 plt.plot(*args: time_est, P_all[:, 0, 0], label="Var(x)", color='1f77b4') # soft blue
222 plt.plot(*args: time_est, P_all[:, 1, 1], label="Var(y)", color='2ca02c') # green
223 plt.plot(*args: time_est, P_all[:, 2, 2], label="Var(v)", color='ff7f0e') # orange
224 plt.plot(*args: time_est, P_all[:, 3, 3] * rad2deg_squared, label="Var(yaw) (deg²)", color='d62728', zorder=1) # red
225 plt.yscale("log")
226 plt.ylim(*args: 1e-2, 1e4)
227 plt.xlabel("Zaman (s)")
228 plt.ylabel("Varyans (log)")
229 plt.title("EKF Kovaryans Bileşenleri (Logaritmik Y Eksenine)")
230 plt.grid(True)
231 plt.legend()
232 plt.tight_layout()
233 plt.show()
```



```

235 # - 13. EKF Hata Analizi: Yönelim (Yaw)  $\pm 3\sigma$  Güven Aralığı -
236 yaw_est_deg = np.degrees(x_estimates[:, 3]) # EKF yaw tahmini (derece)
237 yaw_true_deg = true['yaw'] # Gerçek yaw (derece)
238 yaw_std_deg = np.degrees(np.sqrt(P_all[:, 3, 3])) # Radyan2 → derece cinsinden  $\sigma$ 
239
240 plt.figure(figsize=(12, 5))
241 plt.plot(*args: time_est, yaw_est_deg, label='EKF Yaw Tahmini', color='green')
242 plt.fill_between(time_est,
243                 yaw_est_deg - 3 * yaw_std_deg,
244                 yaw_est_deg + 3 * yaw_std_deg,
245                 color='orange', alpha=0.3, label='±3σ Aralığı')
246 plt.plot(*args: time, yaw_true_deg, label='Gerçek Yaw', color='black', alpha=0.7)
247 plt.xlabel("Zaman (saniye)")
248 plt.ylabel("Yaw (derece)")
249 plt.title("EKF Yaw Tahmini ±3σ Güven Aralığı ile")
250 plt.grid(True)
251 plt.legend()
252 plt.tight_layout()
253 plt.show()

```

